THE PERMUTATION NULL

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
PRECISION MEASUREMENT

ORIOR
2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

For Megan, my loving wife, the only person that really understands what I mean when I speak.

For Autumn and Opal, without you this thought would have never occurred.

To everyone else, I wish you well.

ABSTRACT

Measure how far an object is from the most disordered arrangement of its own parts.

That is the method. Each subject in this work applies it with a different answer to what a part is: atoms in a crystal cell, symbols in a corpus, bytes in a file, coordinates on a circuit board. The reference is built only from the object's own parts. There is neither a prior estimate, a training set nor a model or neural network. The reference is the maximum entropy arrangement of those parts, the shuffled copy that keeps every count, and the measurement is the exact difference from it. In statistics this is a permutation test [5].

Two propositions set the limits of the search. First, any subset of a pattern's positions is a necessary condition for the pattern. A filter that checks a subset never loses a true match. That holds whatever chose the subset, whatever the alphabet is, and whether the positions have an order, as long as each check compares symbols, or compares ranks counted together with the pattern's. Filtering followed by verification is standard in string search [15], and this is the condition under which the filter is lossless. Second, the reverse fails for every proper subset: passing the checks never proves a match. The full comparison cannot be skipped, and every implementation has to verify what its filter lets through. Never losing a match costs nothing. Confirming one costs a full comparison.

The search is implemented in the orior engine, and Section 2.7 of the first chapter states its guarantees, each with the condition it needs. The counts are exact for any valid set of checks on byte data, and on ranked data with 256 classes or fewer under a transitive equality test. Placing the checks greedily drops at least $1 - 1/e$ of what the best placement drops when every alignment is scored [16]. The search goes at most four levels deep, and fewer when the data, the candidates available or the number of checks asked for run out first.

The work uses two basic measurements:

1. **Entropy of the object.** The set T measured against its own maximum entropy. It has no unit. Shannon defines entropy in section 6, page 389, where H is largest, at $\log n$, when every p_i is equal [26].

2. **Difference from the permutation null**, Δ_Σ. How far T is from the maximum entropy arrangement of its own parts. In its bitwise form it lists the exact symmetries of T that the shuffled copies do not reproduce.

The two are not equal as evidence. The shift agreement detector finds a period by how often the data agrees with a shifted copy of itself, and published measurements check its answers. The difference from the permutation null compares the data with shuffles of the same symbols, and no outside source holds the answer to check it against.

The negative result is that collision entropy does not change when the data is shuffled. An object and its most disordered rearrangement have the same symbol counts, and so the same collision entropy. No bound built from a histogram can tell structured data from a rearrangement of the same symbols.

What follows is the simplest form of the algorithm. Other arrangements are in the orior engine workbook [22]. The chapters work through the method as it runs, and the last chapter defines its terms.

CONTENTS

1 SEARCHING FOR A PATTERN, AND WHAT IT COSTS 1

1.1 Finding where a pattern occurs 1

1. Introduction 2
2. Mathematical Framework 2
3. How the measurements were made 7
4. Results 9
5. What the results mean 38
6. Limits and what holds 40
7. Earlier work 42
8. Conclusion 50
9. Data and code availability 51

Appendix A: The hash used as a test oracle 51

Appendix B: The examples 52

2 THE METHOD 55

2.1 Comparing texts by their byte pairs 55

1. Counting byte pairs 55
2. Total variation distance 55
3. Split-half check 55
4. Correcting for how many pairs a text uses 56
5. Text damaged by extraction 56
6. Permutation test 56
7. Checking the tree against the distances 56
8. Sampling one language per family 57

3 CHECKING THE RAREST SYMBOL FIRST 59

3.1 The conjecture 59

4 TERMS 61

4.1 The words this paper uses 61

Sets and arrangements 61

The permutation null 61

Searching 62

Corpora 63

Measurements 63

BIBLIOGRAPHY 65

1

SEARCHING FOR A PATTERN, AND WHAT IT COSTS

1.1 FINDING WHERE A PATTERN OCCURS

One read of one position settles every alignment whose pattern symbol differs from the symbol read. That is $m(1 - 2^{-H_2})$ of the m alignments in expectation, and it agreed to within 0.7% over 84 rows and three ways of choosing the position. The collision probability that sets how many candidates survive therefore also sets how far into a pattern the first mismatch is expected. The same quantity appears once as a rate and once as a length.

The number of candidates a probe finds is an estimator of the Rényi entropy of order two [24], also called the collision entropy. In expectation it equals the plug-in estimator when the probe's own match is counted, and the standard unbiased estimator when it is not. Scored against published estimators on sources whose parameters are known, it is worse in both accuracy and running time at every setting tested. It approached the unbiased estimator as the number of probes approached the length of the corpus.

Its one advantage is what it does without: it builds no histogram and never lists the alphabet. Correcting it means leaving out the case where a position is compared with itself, and applying the same correction again gives the unbiased estimators of the higher Rényi orders. At order two the correction is $(1 - C)/((N - 1)C \ln 2)$ bits for a plug-in collision probability C, which stayed constant across nine sources spanning four orders of magnitude. The numerator is exactly zero when all the probability sits on one symbol. On a source with no entropy, the correction is zero at every order and every length.

Compared with the classic algorithms on the same corpus and the same counter, checking the rarest symbol first loses to a Horspool shift [7] on every corpus and pattern length, by 1.2 to 25 times. A check at offset a can move the search forward by at most $a + 1$ positions, and how far the search moves decides the number of symbol reads. How many candidates survive does not. Treating the two as factors of one product recovers the loss and comes out 11% ahead on C source code. Replacing the frequency model with a running average of how far each offset moves the search, which needs no alphabet, comes out 13.2% ahead, and beats a rule that is given the corpus histogram.

Two negative results apply beyond this search. Estimating how many candidates a sequence of probes leaves by multiplying the probes' individual rates underestimates the count by up to four orders of magnitude on structured data, and always in the direction that leaves too little room. And choosing a probe by the center of mass of a score loses to choosing by the score's maximum on every row measured, by up to 42%.

1. Introduction

A search for a pattern can check every position of the pattern, or it can check the positions most likely to fail first. The second looks like an obvious speedup: check the condition most likely to end the search. The permutation null checks every position of every part of the object, using nothing but the object itself.

Two questions about that search are usually answered together, and they separate when looked at closely. Whether the search can be trusted is a question about its method. How much time it saves depends on the data. This paper keeps the two apart so that each can be tested and checked on its own.

Section 2 covers the method, Section 3 how the measurements were made, Section 4 what was measured, Section 5 what the results mean, Section 6 the limits that were reached, and Sections 7 and 8 earlier work and open questions.

The method is implemented in the orior search engine, and Section 2.7 states what the implementation guarantees. Section 9 says where the software and data are available.

A SHA-256 implementation is used as an instrument. It generates the uniform control corpus and serves as the test oracle that results are checked against. That control is only as strong as its test vectors, and Appendix A covers them.

2. Mathematical Framework

2.0 Definitions

Σ	the alphabet	The set of distinct symbols. It has no topology, metric or order of its own. The only thing it supports is telling two symbols apart: $x_1 = x_2$ or $x_1 \neq x_2$.
δ_0	the comparison	The indicator function of equality. At an offset $d \in G$ it turns a comparison into one bit, match or no match.
∂	the filter	The probe set $A \subset D$. It separates the candidates that are kept from the ones that are dropped.

Checking $\mathrm{Anc}_A(s)$ decides whether an alignment $s \in G$ is kept as a candidate or dropped. Nothing true is lost: every true match is kept, and the full comparison removes the rest. The contrapositive $\neg\,\mathrm{Anc}_A(s) \Longrightarrow \neg\,\mathrm{Occ}(s)$ means that failing one check rules s out of the search for good.

Any choice of $A \subseteq D$ gives a correct search, but the choice decides how fast the search is. How often the symbols at the positions in A occur, and their entropy, decide how many candidates each check drops. That holds when the probes are independent, and Section 4.6 measures cases where they are not. The engine instead chooses probes by how many candidates are left, which captures the correlation between positions that a histogram misses.

The filter is defined only through offsets in G and comparisons in Σ. The search gives the same result under translation, permutation, or a change of coordinates. Section 2.2 guarantees that the count is the same, and Section 4.2 measures it staying the same across geometries. Of the cost, only

rotation has been measured (Section 4.2), and that the cost is the same under the other changes is a conjecture.

2.1 The alphabet and the pattern

Let Σ be the alphabet. Nothing about it is assumed: neither order, metric, finite size nor a way to list its members. The only operation available is a test of whether two positions hold equal symbols, and the test always gives an answer. The engine needs the same, and two things more: that the test depend only on the two positions it is given, and that the caller supply the number of alignments, which the engine cannot work out.

Let $(G, +)$ be an additive group of offsets, and let the data be a map x from a subset of G to Σ. A pattern is a finite set of offsets $D \subset G$, together with the symbols found at those offsets from a chosen starting position. For a starting position t and a candidate position s, define

$$\mathrm{Occ}(s) \iff \forall d \in D : \; x(s+d) = x(t+d)$$

A probe set is any subset $A \subseteq D$, and

$$\mathrm{Anc}_A(s) \iff \forall a \in A : \; x(s+a) = x(t+a)$$

The null used here is the permutation null used throughout this work. The structure of the data defines a topology τ_Σ on the alphabet. Let Π_Σ be the set of shuffles that keep the count of every symbol and any stated grouping. A reference is drawn as $x^\circ_\Sigma = \pi(x)$ for $\pi \in \Pi_\Sigma$, and for a statistic M the quantity measured is

$$\Delta_\Sigma[M] = M(x) - \mathbb{E}_{\pi \sim \Pi_\Sigma}\big[M\big(\pi(x)\big)\big]$$

This adds no new operator and no new code. It is the shuffle that the rest of this work already uses, and the quantity studied is what remains after the shuffle's average is taken away. It does not claim to rebuild a hidden process that produced the data, and it does not rely on Laplace's demon. A model that tried to infer the whole state of a system from its laws would be a different measurement. Here the alphabet gives both the structure and the reference, and $\Delta_\Sigma[M]$ is measured in the object itself.

Neither definition needs G to be the integers, D to be one unbroken run of positions, or Σ to be known in advance.

2.2 Checking a subset of positions never loses a match

For every $A \subseteq D$, all data, and every starting position,

$$\mathrm{Occ}(s) \implies \mathrm{Anc}_A(s)$$

because A is a subset of D, and requiring every position of D to agree includes requiring every position of A to agree. The search uses the contrapositive:

$$\neg\,\mathrm{Anc}_A(s) \implies \neg\,\mathrm{Occ}(s)$$

A position that fails a probe is ruled out exactly and for good. It needs no further check, and no later step looks at it again.

2.3 Passing the checks proves nothing

For every proper subset $A \subsetneq D$, the converse

$$\mathrm{Anc}_A(s) \implies \mathrm{Occ}(s)$$

does not follow, for any data. A counterexample is any s that agrees with the pattern on A and differs at some $d \in D \setminus A$, and such an s can exist whenever the alphabet has more than one symbol. The converse holds for one A only, $A = D$, and then the filter is the full comparison.

A position that passes a probe is only a candidate. No amount of knowledge about the data removes the need for the full comparison, and adding probes leaves fewer candidates without ever making one certain.

2.4 What follows from the two

Any rule for choosing probes is correct. Section 2.2 holds for every subset, whatever chose it. A frequency table, a random choice, or no table at all cannot produce a false negative when the probes compare the symbols themselves. Probes that compare rarity ranks need one more condition, given below. What a choice of probes changes is how often a probe rules a position out.

The alphabet does not have to be known. Both conditions are equalities between positions. When the alphabet cannot be listed, there is no table indexed by symbol, but how rare each symbol is can still be counted using equality alone. That costs up to the square of the data length in equality tests. A rule that counts nothing treats every symbol as equally likely: the maximum entropy case. A table of symbol frequencies is the special case for data whose alphabet is known.

The offsets do not need an order. The definitions use $+$ and never $<$. A grid, a volume, and a rotated set of points differ only in which offsets of G are in D. A rotation rearranges D, and that changes no term in either condition.

Rarity ranks are safe only within one ranking. A probe on ranks is guaranteed to be a necessary condition only when the pattern and the data were ranked together. Which class a position is in comes from the caller's equality test, and it carries over to other data when that test is transitive. The rarity order is counted over one set of data and holds only there. Ranks counted over two different sets can rule out a true match (Section 2.7).

2.5 Expected number of symbol reads

Treat the occurrences of a probe's symbol as independent trials, each succeeding with probability q. Over N candidate positions,

$$\mathbb{E}[C_1] = Nq, \qquad \mathbb{E}[S] = \frac{1}{q}$$

The full comparison sees $\mathbb{E}[C_1]$ candidates, and $\mathbb{E}[S]$ is the stretch a skipping search never reads. Bringing N candidates down to a constant number takes $\log_2 N$ bits of information, and one probe supplies $-\log_2 q$ bits.

The second expression is Kac's lemma [10]: for a stationary ergodic source, the expected time to return to a set of probability q is exactly $1/q$. The gap between hits is therefore a property of the

source instead of the filter. That points to an assumption the method does not supply. Section 2.2 guarantees that no match is lost, this section counts the symbol reads, and neither says that the pattern occurs at all. That depends on the source: a stationary ergodic process reaches any event of positive probability with probability one, and a process that is not stationary or not ergodic makes no such promise. Of these three guarantees, only the first is this paper's.

When a probe's symbol is picked with no information, it is drawn from the data with the data's own frequencies,

$$q \;=\; \sum_{\sigma\in\Sigma} p(\sigma)^2 \;=\; 2^{-H_2}, \qquad H_2 \;=\; -\log_2 \sum_{\sigma} p(\sigma)^2$$

where H_2 is the Rényi entropy of order two [24]. This predicts the number of candidates from the data alone, before any pattern is chosen, and Section 4.4 tests the prediction.

2.6 The filter step by step

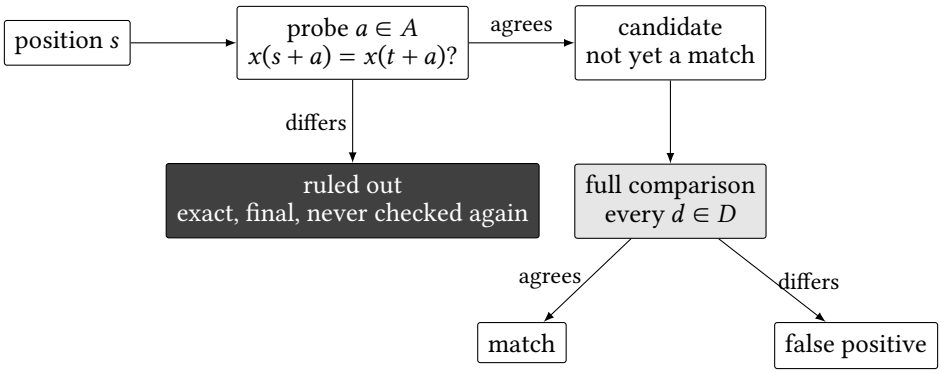


Figure 1. The filter as the benches implement it. A probe settles only one outcome for certain, ruling the position out. The other two need the full comparison, and everything below the candidate box costs time.

2.7 What the implementation guarantees

The results above need no implementation. This section states what the orior engine guarantees, what each guarantee needs, and which claims about the engine do not hold.

The count is exact for any valid set of probes on byte data. Each probe compares the data with the pattern's own symbol at an offset inside the pattern. Each probe is therefore a necessary condition, a set of probes requires all of them, and every alignment the probes leave gets a full comparison. A probe is valid when every position it reads lies inside the pattern. The counting routine that accepts a caller's probes does not check this, and an invalid probe reads past the end of the pattern unchecked. A rejected input returns 0, the same value as a count of zero. An empty pattern matches at every alignment, and every counting routine returns the data length plus one for it. On ranked data the full comparison compares ranks, and the count is exact only where each rank stands for exactly one class (below).

Nothing accumulates, and the engine is not a streaming algorithm. Every comparison is an exact equality. A planning pass marks every alignment as a candidate once, when it starts, and

after that only removes candidates. Every level it reaches is a complete set of probes and gives the same count as any other. Planning can stop at any level. The pass over alignments cannot stop early, because that would lose matches. A planning pass places probes and counts nothing. Its output is the list of candidates, one byte per alignment, which grows with the length of the data. The space bounds proved for streaming pattern matching describe a different model (Section 7.5), and the engine does not sit at a space lower bound.

Probes are placed by greedy coverage. Write R_p for the alignments a probe p rules out. Then $f(P) = |\bigcup_{p \in P} R_p|$ is monotone and submodular. The planner keeps the probe that leaves the fewest candidates, the largest gain in f. By Nemhauser, Wolsey and Fisher [16] the probes it places rule out at least $1 - 1/e$ of what the best set of the same number of single position probes rules out. The bound counts alignments ruled out, and two sets of probes that rule out the same alignments can still cost different numbers of reads. It holds only when every gain is scored exactly, at a sampling stride of one. Where the planner chooses among the caller's own offsets and places all of them, the best set of that size is the whole set, and the ratio holds trivially and guarantees nothing. Stopping when a probe gains nothing is the same as going on only where the set of candidates never grows from one level to the next, and submodularity does not guarantee that.

A planning pass stops, goes a level deeper, rejects its input or runs out of probes, and it never returns to an earlier state. A rejected input returns 0 and writes nothing. An offset already placed is never scored again. A pass with no unplaced offset left stops without applying the stopping rule: asked for four probes for the pattern `ab` in the data `abxbay`, it places two, whether full depth is forced. It returns at most the number of probes asked for, and its depth is at most 4. Below that limit, the number of probes asked for, the offsets available and the data decide how deep a pass goes. The stopping rule compares candidate counts read from the data, and it is overridden only when the caller forces full depth. The depth is not fixed before the program runs. The probe sequences of Section 4, which go to six probes, come from the benches and not from the planner.

Where the reads run. The counting routines are plain loops over one symbol at a time, and so is a planning pass over data of any symbol type, because an equality test hides the representation that a vector compare would read. The only vectorized path is the planner's scan of candidates on byte data at a stride of one. It uses the widest instruction set that the build includes and the processor reports: AVX-512 before AVX2 on x86, SVE before NEON on ARM, and the portable version last. The scan kernels and the exact integer kernels are each implemented for the same six targets: portable, AVX2, AVX-512, NEON, SVE and CUDA. Both kinds compute one operation, an equality at a fixed offset counted over positions. A scan kernel counts the candidates whose byte at one offset is the wanted byte, and an exact integer kernel counts the positions whose value equals the value a fixed lag away. The AVX2, NEON and CUDA scan kernels agree with the portable kernel on every input they have been run on. The AVX-512 and SVE scan kernels have never been run, and are checked only by the instructions in their compiled objects. The CUDA kernel is not selected automatically.

Any symbol type through equality, and ranks only within one ranking. The engine sees a symbol only through an equality test at two positions, and a planning pass works on data of any symbol type through that alone. A projection turns such data into byte ranks, rarest first. A rank combines a class, taken from the caller's equality test, with a place in a rarity order counted over the data that call saw, and that place does not carry over to other data. Ranks from two separate

projections can rule out a true match. Classes are the transitive closure of the equality test over the positions present, at a cost of up to the square of the data length in equality tests. A joint projection ranks the data and the pattern together. A class then gets the same rank on both sides, and no true match is lost under any equality test. The count is exact with 256 classes or fewer under a transitive equality test. With more than 256 classes, or under a test whose closure joins positions that do not agree, the count is an upper bound.

Not built, and open. A reader that the engine could call for more data is described and has no interface. Whether a caller's loop over planning passes can compute anything computable is open. The model named as the target is a tag system [20, 13], no translation into it is written, and the engine as built is finite.

3. How the measurements were made

3.1 The programs that take the measurements

There are two kinds of program, for two kinds of question. The benches test what Section 2 claims, on data built so that the answer is known. The examples measure what real data does, on data that nobody here made.

bench	question	reports
strings	how many candidates pass, and whether probes are independent, on byte strings	candidates, skip, $I(d)$, z, false negatives
geometries	whether Section 2.2 holds beyond a single line	candidates, false negatives
entropy	how good the candidate count is as an entropy estimator	bias, root mean square error
whole search	how a whole search compares with one built another way	symbol reads of the data

The benches are not the engine. They run sequences of up to six probes, and the planning pass of Section 2.7 places at most four.

The examples are 122 programs across nine fields, each one a step of the same pipeline, and every result about real data in the companion research papers comes from one of them. Appendix B lists the steps and how far each field gets.

Nothing is timed. No row is a claim about speed, and every number depends only on the data and the geometry. It is the same on every machine.

3.2 What every row reports

Every row of every bench reports two counts, and they are judged differently.

False negatives. The number of true matches a probe ruled out. Section 2.2 allows one value, zero. A nonzero count is a bug in the bench and never a property of the data: no distribution, geometry or alphabet can break a result that holds for every subset. In the engine, an equality test that depends on more than its two positions and ranks from two separate rankings (Section 2.7) are such bugs. A row that reports a false negative shows a bug to fix and is never read as a result.
Candidates. The number of positions that passed the probes and reached the full comparison. This one is expected to change with the choice rule, the number of probes and the geometry, and Section 4 measures how it changes. It has no correct value.

A case with nothing to check prints `none` instead of zero, so that a row with no work to do cannot be mistaken for a row that did the work and found nothing.

3.3 The byte string corpora

The corpora are 2048 bytes each unless stated otherwise, and are chosen to range from no variety of substrings to as much as possible:

corpus	content	H_2 (bits)
`flat`	one byte repeated	0.000
`structured`	1408 bytes of C source	3.620
`english`	non-repeating English prose	3.692
`periodic16`	16 byte fixed width records, layout repeating and content not	4.611
`uniform`	SHA-256 in counter mode	7.832

Patterns are taken from the corpus. Every one of them occurs. A corpus made by repeating a unit looks the same at the scale of that unit, and then every pattern occurs once per repeat simply because of how the corpus was made. Section 6.5 shows the measurement this spoils. The fill is cut off at the corpus length instead of repeated.

3.4 Rules for choosing probes

There are three rules, as different as the interface allows:

rule	how a symbol is scored
`table`	the cost table built into the bench, which ranks a byte by how rare it is
`random`	a permutation of 0 through 255 drawn from SHA-256, unrelated to frequency
`maxent`	the same cost for every byte. The table carries no information

`random` is a permutation. Every cost is still different and the rule chooses probes exactly as it does with the real table. The only thing taken away is that the table is right. `maxent` makes the rule take the leftmost offsets. The probes sit next to each other. That is the arrangement where multiplying two probes' rates is least accurate.

3.5 Geometries

Six cases share one core, which takes a list of valid starting positions, a list of offsets, and a function that says whether two positions hold the same symbol. The core has no dimension parameter, because the geometry is entirely in the list of starting positions, and no symbol type, because it only asks whether two symbols are equal.

case	G	D
1D line	$\mathbb{Z}$	8 contiguous offsets
2D box	$\mathbb{Z}^2$	a 2 by 4 rectangle
2D scatter	$\mathbb{Z}^2$	8 points in a 5 by 5 window, no row or column filled
2D scatter, turned	$\mathbb{Z}^2$	that scatter under $(r, c) \mapsto (c, 4 - r)$

3D box	$\mathbb{Z}^3$	a 2 by 2 by 2 block
1D complex	$\mathbb{Z}$	8 contiguous offsets over complex symbols
`cube1d` to `cube8d`	$\mathbb{Z}^d$, $d = 1 \dots 8$	a star: the origin, then one step out along each axis in turn

1D complex holds `double _Complex` values made from square roots of primes, and compares them by their stored bytes. Nothing in the file reads a value, orders values, or interprets them.

Probes are chosen by three rules that use no information about the data: the first points, points spaced evenly across the pattern, and a permutation drawn from SHA-256. That file has no cost table and cannot have one, because data whose alphabet cannot be listed has no symbol frequencies to weigh.

3.6 Estimating collision entropy

Each source is written down as a distribution before any data is drawn from it. The true collision probability is therefore known in advance, and no estimator is scored against a histogram of the same corpus it was computed from. There are eight sources: uniform over 2, 16 and 256 symbols; Zipf [35] with exponents 0.5, 1.0 and 1.5; and two sources that put probability 0.90 and 0.99 on one symbol. Corpora are drawn by inverse transform sampling at lengths 256, 1024, 4096 and 16384, with 64 independent trials per row.

Four estimators of the same quantity are scored by bias and root mean square error in bits:

estimator	expression	needs
plug-in	$\sum_\sigma \hat{p}(\sigma)^2$	a histogram
unbiased	$\sum_\sigma n_\sigma(n_\sigma - 1)/N(N-1)$	a histogram
probe, own match counted	mean over probes of matching positions, over N	neither
probe, own match left out	the same, less the probe's own match	neither

The unbiased estimator and the plug-in estimator with its first order bias correction are the same expression. They are one estimator instead of two.

A fifth quantity, the most common value estimate of min-entropy from NIST SP 800-90B section 6.3.1 [17], is computed for comparison and is not scored on the same scale, because min-entropy measures something different.

4. Results

4.0 Summary of the results

The sections below look like a list of separate results, but they are not. One quantity, the collision probability 2^{-H_2}, sets every cost in the method, and there are three conditions under which it stops setting anything. This section says which result is which, so that the rest reads as one argument.

Correctness is not part of it. Sections 4.1 and 4.2 report zero false negatives over 9,396,207 and 213,840 true matches, across three choice rules, thirteen geometries, dimensions one through eight, a rotated set of points, and an alphabet of complex numbers. Section 2.2 does not depend on any distribution. No property of a distribution can affect it. Everything else in this section is about cost.

The cost is one number. Each of these is a function of 2^{-H_2} and the two sizes m and N:

quantity	form	section
candidates per position	2^{-H_2}	4.4
alignments settled by one read	$m(1 - 2^{-H_2})$	4.4.1
estimator bias correction	$(1 - C)/((N-1)C \ln 2)$	4.5.1
fraction a left to right pass collects	$1/((1 + m2^{-H_2})(1 - 2^{-H_2}))$	4.9.4

where the shift stops growing	$3 \cdot 2^{H_2}$	4.9.2
probes before the information runs out	$\log_2 N / H_2$	4.6.1
probe spacing for any order	$mH_2 / \log_2(mH_2 \ln 2)$	4.9.5
candidates in d dimensions	$1 + (P-1)2^{-nH_2}$	4.2

The last row is why the others are worth stating together. Its form holds unchanged from a line to an eight dimensional hypercube, and over an alphabet of complex numbers with irrational parts. That shows the quantity does not depend on the geometry or on what a symbol is.

It stops working in three places, and every failure in this paper is one of them.

The number depends on how the data is read. A corpus has no single H_2, only $H_2(w)$ for the corpus read w bits at a time. Section 4.10 measures it falling from 0.984 to 0.600 bits per bit between $w = 1$ and $w = 16$ on English. Every figure above uses $w = 8$, and nothing here justifies that choice.

A second moment is not a whole distribution. Section 4.3.1 needs the expected minimum of m draws weighted by size, which depends on the whole set of order statistics. Entropy puts the corpora in the wrong order there, and Section 4.8 finds that ranking symbols by the wrong reference costs 25 to 30 times more, which no collision probability predicts.

Independence is assumed throughout, and it fails. Section 4.6 finds the product rule wrong by four orders of magnitude, Section 4.7 finds correlation still present at a separation of seven, Section 4.6.1 predicts 2.98 probes where 6 are measured at $m = 16$, and in Section 4.9.3 the distance rule loses only on periodic data. The collision probability predicts none of these, because each one concerns the joint distribution and the collision probability describes a single position.

4.1 No match lost on byte strings

Over three rules, one to six probes, five corpora, and pattern lengths 1, 2, 3, 4, 16, 64, 256, 1024 and 2048:

All 582 rows pass. 9,396,207 true matches examined. 0 false negatives.

Another 210 rows report `none`. In each of them the pattern has fewer distinct offsets than the number of probes asked for, and no row reports `none` for any other reason.

The sweep includes the extreme lengths. A one byte pattern is the shortest that can hold a probe, a pattern as long as the corpus leaves a single position, and on the flat corpus every position is a match, which gives the most true matches there are to lose.

4.2 No match lost in any geometry

Over three rules that use no information about the data, and one to six probes, on the six geometries above and eight more hypercubes of dimension one through eight:

All 252 rows pass. 213,840 true matches examined. 0 false negatives.

The candidates that pass under rule `shuffled`, against the $N \cdot 2^{-n}$ that a two symbol alphabet predicts:

geometry	n=1	n=2	n=3	n=4	n=5	n=6
predicted	2044.5	1022.3	511.1	255.6	127.8	63.9
1D line	2044.7	1025.7	516.7	259.4	129.2	65.4
2D scatter	1797.8	901.5	449.5	223.1	112.6	56.9
2D scatter, turned	1801.7	902.7	448.6	223.3	112.1	56.7
3D box	1685.3	840.5	422.3	212.7	105.2	52.3
1D complex	2046.4	1023.4	511.9	256.2	128.9	64.7

The grid and cube rows are below the prediction because those geometries have fewer starting positions, 3600 and 3375 against 4089 on the line. Within each row, every added probe halves the count, as predicted.

Rotation makes no difference. 2D scatter and 2D scatter, turned are the same eight points a quarter turn apart, and they agree at every number of probes. That measures the third point of Section 2.4.

Complex numbers behave like bytes. 1D complex matches 1D line to within 0.1% at every count, with the geometry the same and only the symbol type changed.

The cost formula holds in every dimension tested. The hypercube cases put one point of the pattern at the origin and each of the others one step out along the next axis. Each new point of the pattern adds a new axis. The pattern is never a lower dimensional shape sitting inside a larger space. Measured against $1 + (P - 1)2^{-n}$ for P positions and n probes, which counts the one match the pattern always has with itself:

dimension	1	2	3	4	5	6	7	8
positions	4089	3600	4913	4096	1024	729	2187	256
ratio at n=1	1.001	1.000	1.000	1.001	1.001	1.007	0.999	0.995
ratio at n=3	0.991	0.995	1.002	1.000	0.994	1.028	0.999	0.970
ratio at n=6	0.969	1.020	1.017	1.001	0.980	0.996	1.013	0.979

Across all 48 rows the ratio stays between 0.969 and 1.028. Seven points away from the origin can reach at most seven axes. The eight dimensional pattern spans seven of its eight. That is a limit of the pattern's size instead of the method.

The prediction has to include the pattern's guaranteed match with itself. Without it, the ratio drifts to 1.22 on the smallest data: the first pitfall of Section 6.5.

4.3 How much each rule for choosing probes gains

Candidates that pass one probe on patterns of length 16, with the English cost table, fingerprint `69c2e2df`:

corpus	H_2	`table`	`random`	`maxent`	table over maxent
`flat`	0.00	2032	2032	2032	1.00
`structured`	3.62	17.6	47.9	113.3	6.43
`english`	3.69	26.9	108.6	149.5	5.56
`periodic16`	4.61	44.4	96.8	82.3	1.85
`uniform`	7.83	7.53	7.84	7.61	1.01

On `flat` a probe can never fail, and all eighteen combinations of rule and number of probes report 2032 candidates out of 2033 positions. On `uniform` the three rules agree to within 4%. Between those two, a table built from the data's frequencies gains up to 6.4 times.

The order of the last column shows that entropy is not what decides the gain. `periodic16` has more entropy than `english` and less than half its gain, and `structured` has the lowest entropy of the four corpora that are not constant and the second largest gain. Section 4.3.1 derives the quantity that does decide it.

4.3.1 AN UPPER BOUND ON THE GAIN FROM THE RAREST SYMBOL

A probe chosen with a frequency table checks the rarest of the m symbols in the pattern, and a pattern taken from the corpus contains each symbol with probability equal to its frequency. So the rate for the rarest symbol is the expected minimum of m draws weighted by size. Writing

$F(t) = \sum_{\sigma:\, p_\sigma \le t} p_\sigma$

for the size weighted distribution function, the tail sum formula for the expectation gives

$$\mathbb{E}[\min_m] \;=\; \int_0^1 \big(1 - F(t)\big)^m \, dt, \qquad \text{ceiling}(m) \;=\; \frac{\sum_\sigma p_\sigma^2}{\mathbb{E}[\min_m]}$$

The numerator is the rate of a probe chosen with no information, from Section 2.5. The quotient is what a table matched to the corpus gains over no table at all. The ceiling bounds every rule from above, because no rule can pick a symbol rarer than the rarest one in the pattern.

The ceiling is exactly 1 if and only if p is uniform on the symbols that occur. If every nonzero p_σ equals $1/k$, then F is a single step, the integral is $1/k$, and $\sum_\sigma p_\sigma^2$ is also $1/k$. If two nonzero probabilities differ, then for $m \geq 2$ the rarest of m draws is strictly below the size weighted mean and the quotient is greater than one. A source with a single symbol and a uniform source are both uniform on the symbols that occur. Both ends of Section 4.3 report a gain of one, and entropy plays no part in either.

Computed for patterns of length 16 from each corpus's own frequencies:

corpus	H_2	uninformed rate	informed rate	ceiling	measured	captured
`flat`	0.000	1.000000	1.000000	1.000	1.00	exact
`structured`	3.620	0.081313	0.004483	18.14	6.43	35%
`english`	3.692	0.077371	0.011887	6.51	5.56	85%
`periodic16`	4.611	0.040927	0.013708	2.99	1.85	62%
`uniform`	7.832	0.004389	0.002172	2.02	1.01	50%

The ceiling puts the corpora in the right order where entropy does not, and it is exact for the single symbol source.

The gap between the ceiling and the measurement has two separate causes. On `english` the cost table is the English table, and 85% is the largest share of the ceiling reached anywhere in this paper; that is the effect of a table that matches the data. On `structured` the same table reaches 35% of a much larger ceiling: the cost of a table that does not match.

The `uniform` row has a different explanation. Its ceiling of 2.02 is not a property of the source, which is uniform and so has a ceiling of exactly 1 by the statement above. It is a property of the sample: 2033 draws from 256 symbols do not come out exactly even, and that unevenness is something a rule that knew the sample could use. No fixed table can, because the unevenness is not part of the source and a table cannot know which symbols this particular corpus happened to draw too rarely. The measured 1.01 is the size of that limit.

4.4 Predicting the candidate count from collision entropy

The predicted number of candidates under `maxent` against the measured number, with one subtracted from each prediction because the rows leave out the pattern's own match:

corpus	H_2	predicted	measured	error
`flat`	0.000	2033	2032	0.05%
`structured`	3.620	112.3	113.3	0.9%
`english`	3.692	156.3	149.5	4.3%
`periodic16`	4.611	82.2	82.3	0.1%
`uniform`	7.832	7.92	7.61	4.0%

The remaining error is a sampling bias in how the patterns are drawn. Patterns are taken at a fixed spacing. In English the probed byte is a biased sample of letters, and the two rows with the largest error are the two where that spacing lines up with the corpus's structure.

4.4.1 HOW MANY ALIGNMENTS ONE READ RULES OUT

Sections 2.2 and 4.1 treat a failed probe as ruling out one position. In fact it rules out many.

Checking the probe at pattern offset a reads the data at $s + a$ and finds some symbol c. For any shift δ, the alignment that starts at $s + \delta$ puts pattern offset $a - \delta$ on that same position, and that alignment can match only if $v_{a-\delta} = c$. So one read rules out every shift whose pattern symbol differs from the one found, and their number is

$$R = m - \left|\{\, i : v_i = c \,\}\right|$$

A symbol that is not in the pattern rules out every alignment that covers the position. Taking the expectation over a corpus, with the pattern drawn from that corpus so that c occurs with its own frequency,

$$\mathbb{E}[R] = m\Big(1 - \textstyle\sum_\sigma p_\sigma^2\Big) = m\big(1 - 2^{-H_2}\big)$$

The quantity that sets the candidate rate in Section 4.4 also sets how far into a pattern the first mismatch is expected. It appears once as a rate and once as a length. Measured at every pattern length on every corpus:

corpus	m=4	m=16	m=64	m=256	ruled out over m
`english`	0.9962	0.9997	1.0000	1.0002	0.923
`structured`	0.9939	0.9991	0.9985	0.9934	0.917
`periodic16`	0.9999	0.9999	1.0001	1.0001	0.959
`uniform`	1.0001	1.0000	1.0000	1.0000	0.996
`flat`	exact	exact	exact	exact	0.000

Each entry is measured over predicted. This rule gives 35 rows, of which 28 have a ratio, and it stays between 0.9934 and 1.0011. Over all three rules, 84 rows have a ratio, and the range is 0.9932 to 1.0011. On `flat` both sides are exactly zero at every length, because every position holds the one byte the pattern is made of, and no read can rule anything out.

The last column does not change with m within a corpus, as the expectation requires: the share of alignments one read rules out is $1 - 2^{-H_2}$, which depends on the corpus alone. One read rules out 92% of the alignments that cover it on English prose, and 99.6% on a uniform corpus.

This is the bad character shift of classic string search [7], derived from Section 2.2 instead of from a shift table, with its size set by the collision probability. Nothing in the derivation uses an order on positions. The result carries over to the geometries of Section 4.2, though it was measured only on byte strings.

4.5 The candidate count as an entropy estimator

Bias and root mean square error in bits, over 64 trials with 64 probes each:

source	N	true H_2	plug-in	unbiased	probe, own match counted	probe, own match left out
`uniform256`	256	8.000	-0.984 / 0.986	+0.029 / 0.126	-0.964 / 0.969	+0.074 / 0.210
`uniform256`	1024	8.000	-0.325 / 0.325	-0.005 / 0.029	-0.324 / 0.335	-0.003 / 0.108
`uniform256`	4096	8.000	-0.088 / 0.089	-0.001 / 0.007	-0.077 / 0.085	+0.010 / 0.040
`zipf1.0`	4096	4.515	-0.008 / 0.061	-0.001 / 0.061	-0.050 / 0.204	-0.042 / 0.203
`skew0.99`	4096	0.029	-0.000 / 0.005	-0.000 / 0.005	+0.000 / 0.019	+0.000 / 0.020

When the probe counts its own match it reproduces the plug-in estimator, and when it leaves its own match out it reproduces the unbiased estimator. At `uniform256` and $N = 256$, the plug-in bias is -0.984 bits and the probe counting its own match gives -0.964; the unbiased bias is $+0.029$ and the probe leaving it out gives $+0.074$. Subtracting the probe's own match, which Section 6.5 treats as a measurement error, is the standard bias correction arrived at from the other side.

The probe is worse than both published estimators at every setting tested. Its error is set by the number of probes instead of the corpus length. Root mean square error at $N = 16384$, `zipf1.0`:

number of probes	16	64	256	1024	4096
probe, own match left out	0.602	0.252	0.122	0.062	0.039
unbiased	0.028	0.028	0.028	0.028	0.028
ratio	21.7	9.1	4.4	2.2	1.4

The probe's error falls as one over the square root of the number of probes, and it approaches the unbiased estimator without reaching it. The probe also costs more, $O(kN)$ against $O(N)$ for a histogram. Its one advantage is what it does without: it builds no histogram and never lists the alphabet. It works where the other two cannot be computed at all. Inside a search it costs nothing, because the candidate count comes out of work the search is already doing.

4.5.1 BIAS CORRECTION BY FALLING FACTORIALS The correction stops a position from being paired with itself. Applied to k-tuples, it leaves out every tuple in which two indices are the same, which puts a falling factorial in the numerator and the denominator and gives the unbiased estimator of $\sum_\sigma p(\sigma)^k$. So the correction extends order by order, and each order estimates the Rényi entropy of that order.

At order two the correction has a closed form. Writing $C = \sum_\sigma \hat{p}(\sigma)^2$ for the plug-in collision probability, the corrected value is $(CN - 1)/(N - 1)$. The two differ by

$$\Delta C \;=\; \frac{1 - C}{N - 1}, \qquad \Delta H_2 \;\approx\; \frac{1 - C}{(N - 1)\, C\, \ln 2} \text{ bits}$$

The numerator answers the question the higher orders were computed for. A source with a single symbol has $C = 1$ exactly. $1 - C$ is exactly zero, and the correction is exactly zero at every length. The same holds at every order: with one symbol, $n_\sigma = N$, the plug-in form gives $\sum \hat{p}^m = 1$ and the corrected form gives $N^{(m)}/N^{(m)} = 1$. Measured at orders 2 through 5 and lengths 256, 1024, 4096 and 16384, every entry is 0.0000 bits.

Size of the correction in bits at order 2, $N = 4096$, 64 trials, against the closed form:

source	H_2	predicted	measured
`point1`	0.000	0.0000	0.0000
`skew0.99`	0.029	0.0000	0.0000
`skew0.90`	0.286	0.0001	0.0001
`uniform2`	1.000	0.0004	0.0004
`zipf1.5`	2.364	0.0015	0.0015
`uniform16`	4.000	0.0053	0.0053
`zipf1.0`	4.515	0.0077	0.0077
`zipf0.5`	7.254	0.0534	0.0525
`uniform256`	8.000	0.0898	0.0871

The formula holds across four orders of magnitude in the size of the correction. It is furthest off at `uniform256`, where the first order approximation of the logarithm is weakest.

The size of the correction depends on how many times each symbol was seen, because a falling factorial differs from a plain power in proportion to that count. A common symbol seen 4055 times out of 4096 loses almost nothing when it is not paired with itself. A symbol seen 16 times loses a measurable share.

Going to higher orders does not settle on one behavior, and the direction depends on where a source puts its probability. On a spread out source the correction grows with every order, because small counts stay small at higher order: `uniform256` gives 0.0871, 0.1256, 0.1616 and 0.1955 at orders 2 through 5. On a skewed source it shrinks, because higher moments concentrate on the most common symbol, where the count is large: `zipf0.5` gives 0.0525, 0.0393, 0.0327 and 0.0331. On the single symbol source it is zero at every order.

This correction and the ceiling of Section 4.3.1 depend on different properties of a distribution, and neither predicts the other. The ceiling is one when the probabilities of the symbols that occur are equal, at any entropy and any corpus length. The correction is zero only for a single symbol source, and its size is set by how many times each symbol was seen. The correction falls as the corpus grows, and the ceiling does not. A uniform source and a single symbol source both have a ceiling of one, and their corrections are 0.0871 and 0.0000 bits at $N = 4096$. The two were measured on different sets of sources, and no sweep over both was run.

4.6 Dependence between probes in a sequence

The ratio of measured candidates to the number predicted under independence, for patterns of length 16:

corpus	rule	n=2	n=3	n=4	n=5	n=6
`uniform`	`table`	1.23	-	-	-	-
`english`	`table`	1.44	3.80	7.68	160.7	-
`structured`	`table`	8.46	163.2	663.4	2226.5	8186.1
`english`	`maxent`	2.20	12.1	77.4	438.4	2409.4
`structured`	`maxent`	2.34	8.53	28.9	118.0	389.7

A dash marks a cell where no candidates were left.

Multiplying the rates promises too much, and the error grows with every probe added. On `structured` with the frequency table, it is wrong by three orders of magnitude at four probes and by four at six. The error is in the dangerous direction: more candidates are left than the arithmetic predicts. On `uniform` the ratio stays at 1.23 and no candidates are left after three probes. That is the case where the arithmetic is exact.

4.6.1 HOW MANY PROBES, IF THEY ARE INDEPENDENT Section 2.5 counts the cost of a search in bits: bringing N candidates down to a constant number takes $\log_2 N$ bits, and one probe supplies $-\log_2 q$ of them. A probe chosen with no information supplies H_2 bits. n probes use up the information at

$$n = \frac{\log_2 N}{H_2}$$

Compared with the first number of probes at which the measured count of extra candidates reaches zero:

corpus	H_2	N	predicted	measured, $m = 64$ and 256	measured, $m = 16$
`uniform`	7.832	2048	1.40	3	3
`periodic16`	4.611	2048	2.39	3 to 4	4
`structured`	3.620	1408	2.89	beyond 6	beyond 6
`english`	3.692	2048	2.98	3	6
`flat`	0.000	2048	unbounded	never	never

The prediction assumes that every probe supplies its full H_2 bits, which needs them to be independent. Section 4.7 finds that they are at a separation of 7 or more. At $m = 64$ and above there is room for that, and the bound is exact on English, predicting 2.98 against a measured 3. At $m = 16$ there is not: the probes are close enough to be correlated, each one supplies less than H_2 bits, and English needs 6.

The gap between the predicted and measured number of probes is the failure of independence from Section 4.6, seen another way. `structured` has both the largest ratio in Section 4.6 and the largest gap. `flat` supplies no information at all. No number of probes rules anything out, and the row that never finishes is the same corpus that every other measurement reports zero on.

In the benches, a sequence of probes leaves no extra candidates after three probes on `uniform`; after three or four on `periodic16` at $m = 64$ and 256, and four at $m = 16$; and after three on `english` at $m = 64$ and 256, and six at $m = 16$. `structured` still has extra candidates after six, and `flat` always does. Where a sequence finishes, it finishes because the information is used up. The engine's planning pass is a different program. It stops at four levels, and the number of probes asked for, the offsets available and the data decide how deep it goes (Section 2.7).

4.7 Dependence by distance and by period

Dependence $I(d)$ against the distance between two probes, for patterns of length 64, rule `table`:

stride	1	2	3	5	7	8	13	17
`english`	6.05	2.59	1.50	1.21	0.99	1.02	1.07	1.21
`structured`	6.20	3.70	3.79	2.88	1.46	1.54	1.07	1.17

Correlation depends on distance, not on pattern length, and is gone by a distance of 7 in both corpora. The `uniform` control is reported as z, not as I: its extra joint count runs from 0.00 to 0.09 against a prediction of 0.03, and the ratio swings between 0.00 and 3.03 on counts too small to support a ratio. Every $|z|$ in that row is at most 1.7, which is consistent with independence at every distance measured.

On `periodic16`, reported as z:

stride	3	7	11	16	17
length32	0.9	**4.0**	-0.7	**5.3**	0.7
length64	-0.3	**6.6**	-0.1	**12.3**	0.4
length128	-0.1	**6.5**	0.7	**13.1**	0.5
length256	-0.4	**5.5**	0.7	**12.6**	1.4

A distance equal to the record length gives the strongest correlation measured anywhere in this paper, $I = 5.82$ at $z = 12.3$, and it appears at every pattern length. A distance of 17, which shares no factor with 16, shows none.

$I(d)$ is the exponential of the pointwise mutual information: $\mathrm{PMI}(d) = \log_2 I(d)$, and the second probe supplies $-\log_2 p_\beta - \mathrm{PMI}(d)$ bits. At $I = 5.82$ that is 2.54 bits less. A probe that should supply 8 bits supplies 5.5.

For fixed width records of length T with fields at offsets F, the record layout predicts that correlation can occur only at

$$D_x \subseteq \{T\} \cup \{\, |f_i - f_j| \;:\; f_i, f_j \in F \,\}$$

Here $T = 16$ and the punctuation is at $F = \{4, 11, 15\}$. $\hat{D} = \{4, 7, 11, 16\}$. A distance counts as correlated when $|z| > 2$. Distances 1 and 2 are left out, because short range correlation is present in every corpus here, English included, at $I(1) = 6.05$.

d	in $\hat{D}$	$I(d)$	z	measured	verdict
3	no	0.95	-0.3	independent	agrees
4	**yes**	0.96	-0.3	independent	**necessity fails**
5	no	1.31	2.1	correlated	**sufficiency fails**
7	**yes**	2.46	6.6	correlated	agrees
8	no	1.64	4.1	correlated	**sufficiency fails**
11	**yes**	0.99	-0.1	independent	**necessity fails**
13	no	1.13	0.9	independent	agrees
16	**yes**	5.82	12.3	correlated	agrees
17	no	1.08	0.4	independent	agrees

Five of the nine agree, and the four that do not are the result. The $d = 5$ row is at $z = 2.1$, against a threshold of 2.0.

The prediction misses the independence at $d \in \{4, 11\}$ because a predicted distance only matters when the rule chooses probes at both of its ends, and the cost table decides which probes are

chosen. It misses the correlation at $d = 8$ because $8 = T/2$, and a periodic sequence is correlated at the divisors of its period, which the formula as written leaves out. Adding those divisors would catch that, but it would enlarge $\hat{D}$ and make the first kind of miss worse. So $\hat{D}$ bounds the distances at risk in neither direction, and the only rule that holds is the weak one: a distance is safe when it has been measured to be safe.

One confounding factor: the probe is chosen by the English cost table on data that is not English. These rows measure the mismatch between table and data as well as the period.

4.8 Using the wrong frequency table

$I(d)$ and the skip both measure a difference from a reference distribution, and the cost table is that reference. A build includes exactly one of five tables. Skip distance for patterns of length 64, at a distance of 0:

corpus	english	generic	uri	route
english prose	159.1	**168.0**	5.8	6.0
C source	147.7	**206.7**	5.9	6.8
periodic records	73.2	73.2	16.6	9.5
uniform (control)	277.1	256.0	277.7	277.1

The uniform row of this table was measured with a different uniform generator from the one in Section 3.3. What the row shows, that the four tables agree on uniform data, does not depend on which uniform source is used.

The wrong reference costs 25 to 30 times. `uri` and `route` bring the skip on real text from about 160 down to about 6. That is the largest effect measured anywhere in this paper.

The prediction that a table matched to the data does best is false. `generic` beats `english` on English prose, and by more on C source. So the reason to choose the table carefully is not that it should match the data, which the measurements do not support. The reason is that the wrong table costs 25 times, and a build holds only one.

On the uniform corpus every table does about the same, at 277.1, 256.0, 277.7 and 277.1. At maximum entropy the choice of reference Q stops mattering, because $D(P\|P) = 0$ [12]. Section 4.3 reaches the same conclusion from the other side, by removing the reference instead of replacing it, and again the uniform corpus shows no difference.

4.9 A left to right search against probe filtering

Sections 4.3 to 4.6 measure candidate rates, which is not the same as the work a whole search does. The whole-search bench runs every method on one corpus, with one set of patterns and one counter, the number of symbols read from the data, and checks every method's matches against a brute force scan. A row where the methods disagree prints BROKEN and is not used.

4.9.1 THE COST OF PROBING THE RAREST SYMBOL

A probe at offset a can move the search forward by at most $a + 1$, because beyond that the pattern no longer covers the position read, and the read says nothing. Horspool [7] probes at $m-1$, the largest offset a pattern has. That gives the largest possible shift and the worst candidate rate. Probing the rarest byte gives the best candidate rate and gives up shift.

Symbols read from the data per search, mean over 64 patterns, English cost table:

corpus	m	Horspool	rarest byte	rarest byte, random offset	rarest over Horspool

english	16	361.0	671.8	694.9	1.86
structured	16	170.3	363.0	332.8	2.13
periodic16	16	360.1	1551.2	859.1	4.31
uniform	16	281.6	3517.0	720.5	12.49

Probing the rarest byte loses on every corpus and every pattern length tested, by 1.2 to 25 times, both on the mean and on the worst of the 64 patterns. Randomizing the offset does not recover it. Adding the rarest byte as an extra filter on top of Horspool gains between 0.0% and 2.6%.

The loss is largest on `uniform`, where no byte is rarer than the others and the cost table picks an offset unrelated to the data, and the largest possible shift falls with it. The cost of a search is decided by how far it shifts, and the candidate rate is not the limiting factor.

This comparison can only be made on a line. Horspool needs a total order, so that a pattern has a last position, and a translation along it to shift by, and neither exists for the sets of positions in Section 4.2. The comparison measures the cost of the general method where the extra structure is there to use.

4.9.2 RULING OUT AND SHIFTING TOGETHER

How often a probe rules a position out and how far the search then shifts are two factors of one quantity. A read at offset a that returns symbol c shifts the search by $\mathrm{adv}_a(c)$. A probe is worth

$$\mathrm{E}[\text{advance}] \;=\; \sum_c p(c)\,\mathrm{adv}_a(c), \qquad \mathrm{adv}_a(c) \le a+1$$

Horspool takes the largest possible shift and ignores frequency. Probing the rarest byte takes the best rate of ruling out and ignores the limit on the shift. Choosing the offset that maximizes the product, using the corpus's own frequencies:

corpus	m	chosen offset of $m-1=31$	vs Horspool
uniform	32	31.00	1.0000
english	32	28.97	0.9692
structured	32	29.73	0.8902
periodic16	32	30.66	1.0273

On `uniform` it chooses $m-1$ at every pattern length and ties Horspool to four decimal places. On C source it gives up two positions of possible shift to reach a rarer byte, and wins by 11%. This method reads the frequencies of the whole corpus before searching, and that cost is not counted. It is an upper limit on how well a probe can be chosen, not a rule that can be used.

Which of the two happens is decided in advance by one number. If the pattern holds the symbol read at rate $q = 2^{-H_2}$ per position, the distance to its nearest earlier occurrence has $\Pr[d > j] = (1-q)^j$, so

$$\mathrm{E}[\text{advance}(a)] \;=\; \sum_{j=0}^{a}(1-q)^j \;=\; \frac{1-(1-q)^{a+1}}{q}$$

This rises and levels off at $1/q = 2^{H_2}$, passing 95% of that level near $a = 3\cdot 2^{H_2}$. Below that distance the expected shift is still growing, the largest offset wins, and a rarer symbol cannot make up for the shift it gives up. Above it, the expected shift is flat and rarity alone decides.

The chosen offset over $m-1$, against that distance, with the column of the predicted crossing:

corpus	$3\cdot 2^{H_2}$	8	16	32	64	128	256
uniform	683.6	1.000	1.000	1.000	1.000	1.000	0.998
periodic16	73.3	1.000	0.996	0.989	0.974	0.888	0.770
english	38.8	0.993	0.975	0.935	0.911	0.869	0.774
structured	36.9	0.993	0.968	0.959	0.963	0.913	0.893

On `uniform` that distance is longer than any pattern tested. No pattern reaches the flat part, and the best offset stays at $m-1$ throughout, to three decimals at five lengths of six. `periodic16` crosses between 64 and 128, and its chosen offset falls from 0.974 to 0.888 across exactly that

interval. The two corpora whose distance is near 37 have already started to drift while still below it. $3 \cdot 2^{H_2}$ is the right scale instead of a sharp boundary.

4.9.3 ESTIMATING THE SHIFT WHILE SEARCHING A frequency table needs an alphabet that is finite and can be listed, which Section 2.1 does not assume. What stays finite when the alphabet does not is the shift, because a shift is a number of positions. So the method below keeps one running average of the shift for each offset, m numbers, and never deals with symbols at all.

Two of its properties follow from how it is built instead of from tuning. Each offset starts at $a + 1$, the largest shift it can reach. The method starts as Horspool at the last offset and can only revise its estimates downward as reads come in. And one read updates every offset, because the shift another offset would have got from the same position depends only on where the pattern holds the symbol read, which one backward pass over the pattern computes without any table of symbols.

corpus	m	Horspool	product rule, given the histogram	running average, given nothing
`structured`	32	136.6	121.6	**118.5**
`structured`	64	129.5	128.9	**121.7**
`structured`	4	445.2	445.2	**426.0**
`structured`	128	177.6	170.2	**168.3**
`english`	128	274.4	255.7	266.4
`uniform`	all			1.000 to 1.008 of Horspool

The running average beats Horspool on 15 of 28 rows, by up to 13.2%, and beats the product rule on every `structured` row, even though the product rule is given the corpus histogram and the running average is not. The product rule works out the shift from one frequency model of the whole corpus. The running average measures the shift directly and forgets old reads at a rate of 0.1. It follows local structure that a histogram of the whole corpus averages away.

Updating only the offset that was read, instead of all of them, leaves the running average level with Horspool (0.95 to 1.05). The gain comes from updating the offsets that were not read. Adapting does not supply it.

That update makes an assumption, and the one corpus where the method loses breaks it. A read at offset a returns the symbol at position $s + a$, and offset a' is credited with the shift that symbol would have given it. But the position offset a' would actually have read is $s + a'$. The update is right only where the distribution of symbols does not depend on position. Fixed width records make it depend on the column, and the two positions are in different columns.

The running average over Horspool, by pattern length:

corpus	4	8	16	32	64	128	256
`periodic16`	1.000	1.003	**1.050**	**1.056**	1.024	1.006	0.953
`structured`	0.957	0.946	0.908	0.868	0.940	0.947	0.992
`english`	1.002	1.003	0.998	0.994	0.985	0.971	0.981
`uniform`	1.000	1.000	1.004	1.005	1.008	1.004	1.000

`periodic16` is the only corpus above one over a run of lengths, and the loss is largest at 16 and 32, the record length and twice it, falling off on either side. The three corpora with no columns show nothing. So the same mechanism gains 13.2% in one place and loses 5.6% in another, and what decides which is whether position carries information that the update ignores.

Reading the second position instead of inferring it would remove the assumption at the cost of one more read, the same read the update exists to save. That trade-off is not measured here.

4.9.4 HOW MUCH A LEFT TO RIGHT SEARCH USES OF WHAT IT READS Section 4.4.1 counts the alignments one read could rule out, and Section 4.9.1 counts the reads a left to right

search makes. Those two are enough to find the cost of going left to right without measuring anything more.

A read that returns symbol c rules out every alignment whose pattern position holds a different symbol, $m(1-2^{-H_2})$ of them in expectation. A search that must go left to right can only take the unbroken run of ruled out alignments up to the nearest one that is still possible. It stops at the nearest occurrence of c in the pattern. That symbol occurs about $m\,2^{-H_2}$ times. The nearest one is about $m/(1+m\,2^{-H_2})$ away. The fraction of the available information a left to right shift uses is

$$\text{harvest} \;\approx\; \frac{1}{\left(1+m\,2^{-H_2}\right)\left(1-2^{-H_2}\right)}$$

Compared with the reads already reported, with no fitted parameter. The measured column is a lower bound, because the read count includes the full comparisons and so understates the mean shift:

corpus	m	H_2	predicted	measured at least
structured	16	3.620	47.3%	47%
english	16	3.692	48.4%	52%
periodic16	16	4.611	63.0%	74%
uniform	16	7.832	93.9%	91%
english	64	3.692	18.2%	19%
english	256	3.692	5.2%	3%

At a fixed m the fraction rises with H_2: 47%, 52%, 74% and 91% on the four corpora in order of entropy. It falls sharply as m grows, because a longer pattern holds more copies of whatever symbol was read.

What a search gains by dropping the order. For large m the fraction tends to $2^{H_2}/m$. The most that dropping the left to right order can gain is $m\,2^{-H_2}$, the expected number of times the symbol read occurs in the pattern. That is about 20 on English prose at $m = 256$, and 1.07 on the uniform corpus at $m = 16$. Going in order costs something when the entropy is low and the pattern is long, and there is nothing to gain when the entropy is high and the pattern is short.

By Section 2.2, every alignment a read rules out is ruled out exactly and for good, and none of that information is lost by being ignored. A left to right shift leaves between 3% and 99% of it unused, and the table above gives the figure for each row.

4.9.5 PROBING IN ANY ORDER

Section 4.9.4 measures how much a left to right search leaves unused. This section uses it.

If the order is free, a read does not belong to any probe. One read of one position rules out every alignment whose pattern position holds a different symbol. The only choice is which positions to read, and those can be fixed in advance. Reading every kth position covers each alignment m/k times. With $x = m/k$ the candidates left are $N2^{-H_2 x}$, the probe reads are Nx/m, and the total is smallest at

$$x \;=\; \frac{\log_2(mH_2 \ln 2)}{H_2}, \qquad \text{reads} \;\approx\; \frac{N}{m}\left[x + \frac{1}{H_2 \ln 2}\right]$$

This model predicts savings of 11 to 26 times at $m = 256$. The measured savings are an order of magnitude smaller, and the reason is the useful part.

Measured as left to right reads over reads in any order, so that above one is a saving:

corpus	$m = 4$	$m = 16$	$m = 64$	$m = 128$	$m = 256$
english	0.96	0.85	1.38	**1.47**	1.33
structured	1.07	0.81	1.13	1.16	1.08
periodic16	0.63	0.76	0.99	1.20	1.36
uniform	0.44	0.27	0.93	0.94	0.98

mixed	0.66	0.82	0.80	0.93	1.05

The dependency depth is 2 on every row, as the method requires. It is the depth of the branch-free search, where every probe is issued at once and one branch is taken on the combined result, and it has nothing to do with how many levels a planning pass goes through (Section 2.7).

The probing step behaves exactly as modeled, and the model still fails. The candidates left after probing at $m = 256$ are 1, 7, 8, 11 and 2 on the five corpora, against a prediction of 1.25 to 3.84. The reads are going somewhere the model does not count.

Every pattern here is taken from the corpus it is searched in. Every search has to confirm one true match, and confirming a match of length m takes m reads. The derivation above leaves that term out. Adding it accounts for the measurements to within 3%:

corpus	probes	$+m$	+ false candidates	predicted	measured
english	25.1	256	9.0	290.1	289.7
structured	9.7	256	0.0	265.7	267.0
periodic16	31.5	256	10.5	298.0	296.0
uniform	19.8	256	15.0	290.8	287.6

A long search is limited by the full comparison instead of the filter. Any algorithm that must confirm the match needs at least m reads. At $m = 256$, Horspool takes 1.10 to 1.57 times that minimum and probing in any order takes 1.04 to 1.16. Both are already close to a limit that no filter can move. An order of magnitude was never available. The saving that does exist is the gap between those two ratios, and the largest measured is 1.47 at $m = 128$ on English.

On short patterns, probing in any order loses, badly on the uniform corpus at 0.27, because reading at a spacing shorter than the pattern costs more reads than a shift does, and there is no minimum from the full comparison to hide the cost.

The bookkeeping is bitwise, and that makes it cheap. The alignments a read rules out form a set, sets combine by union in any order by Section 2.2, and a set of alignments is a bit vector. Adding a read costs $\lceil m/W \rceil$ word operations for a machine word of W bits, the candidates left are the complement, and no other structure is needed. At $m = 256$ with 64 bit words the whole probing step is 25 reads and 100 word operations, at a dependency depth of two, against 385 reads that must run one after another.

Each mask is indexed by pattern position. It is m bits wide whatever the alphabet and whatever the dimension, and the vector of alignments the masks clear grows with the data (Section 2.7). So Section 2.1 can leave the alphabet and the dimension unbounded at no cost: neither appears in what the search stores.

4.9.6 THE FULL COMPARISON CANNOT BE SKIPPED

Section 4.9.5 finds that a long search is limited by the full comparison, and the obvious question is whether the full comparison can be dropped. By Section 2.2 every match passes every set of probes. If the candidates are as few as the matches, they are exactly the matches and confirming them tells nothing new. The method below probes at the spacing that leaves one candidate in expectation, reports the candidates as matches, and never confirms them.

It is cheap, and it is wrong.

corpus	m	in-order reads	unconfirmed reads	saving	searches wrong, of 64
structured	256	287.5	14.0	20.5	51
english	256	384.9	34.0	11.3	32
uniform	256	281.6	25.0	11.3	64
structured	128	177.6	27.0	6.6	49
english	4	916.5	2789.0	0.33	0

The last row is the only kind that is ever right: at a spacing of one, every position is read, no false candidate can be left, and the cost is higher than reading the corpus straight through. Everywhere else the saving is real and the answer is wrong on half to all of the searches.

Both results of Section 2 show up here as measurements. Across all 35 rows, the number of searches that report too many matches equals the number that are wrong. Not one search, on any corpus, at any pattern length or spacing, reports fewer matches than there are. Section 2.2 is otherwise an argument, and here it shows in data. Section 2.3 shows in the same column: the candidates always include more than the matches, and probing does not close the gap short of reading everything.

The error goes in one direction only, and that makes it a signal. Since missing a match is impossible, any difference is an overcount. An overcount can be detected without knowing the answer, and it means exactly one thing: the spacing is too wide. The change is sharp. On English at $m = 4$, going from a spacing of 1 to 2 takes the wrong searches from 0 to 61 of 64.

A spacing tuned on one known pattern is not safe for others. A signal that goes in one direction and grows with the spacing can be searched by bisection. One pattern whose number of matches is known can find the widest spacing that still gives that number, in $\log_2 m$ passes. Using that spacing on the other 63 patterns:

corpus	m	calibrated	derived	wrong of 63	over in-order reads
`structured`	256	98	91	23	22.1
`english`	256	75	84	28	10.1
`uniform`	256	145	170	59	9.7
`periodic16`	256	66	99	1	6.4
`uniform`	16	9	11	62	0.6
`english`	8	1	3	0	0.19

Every row with no errors is a row where tuning pushed the spacing down to one or two, which reads nearly every position and costs more than a left to right search. Every row that is fast is also wrong on a third to nearly all of the patterns it was not tuned on.

The tuned spacing and the spacing from the formula agree closely: 98 against 91, 75 against 84, and 44 against 45. So the failure is not a wrongly set constant. The safe spacing depends on the pattern and the corpus together: a pattern with rare symbols allows a wide spacing and one with common symbols does not, and a measurement on one pattern says nothing about another. One known count tunes one pattern.

Where the model is wrong. The model's 11 to 26 times rests on the independence assumption that Section 4.6 finds failing worst on `structured`, so that is the obvious place to look for its error. The error is not there. Independence holds here: 1.25 to 3.84 candidates predicted against 1 to 11 measured. What the model leaves out is the m reads needed to confirm the match every pattern is guaranteed to have, and that term has nothing probabilistic in it.

A model can be wrong in a place no one is watching. The weakest stated assumption here is not where the error is; it is in an assumption that is not stated at all.

4.9.7 CHOOSING EACH PROBE FROM THE READS SO FAR

Section 7.1 separates two approaches. A fixed set of probes can be made safe by working it out from the pattern in advance, as deterministic sampling does. Sections 4.9.5 and 4.9.6 measure a fixed set of probes without that step, and it fails. A third approach holds no fixed set of probes at all.

The rule needs no analysis of the pattern and no distribution. The symbol about to be read is unknown. Every position rules out the same expected fraction of the alignments that cover it, and what differs between positions is how many remaining alignments cover them. So the next read

goes where the remaining candidates overlap most, which depends only on the answers so far. The rule stops when a read would rule nothing out, and then confirms what is left. It scores each read by a fraction expected before the read is made. The $1 - 1/e$ bound that Section 2.7 states for the engine's exactly scored planning pass is not known to hold for it.

corpus	reads at $m = 256$	probes among them	reads over the floor	in-order over the floor
`structured`	262.1	9.5	1.02	1.12
`uniform`	277.7	23.5	1.08	1.10
`english`	279.4	26.6	1.09	1.50
`mixed`	281.8	28.7	1.10	1.21
`periodic16`	307.4	45.4	1.20	1.57

The minimum is the m reads that Section 4.9.5 shows cannot be avoided. This rule comes within 2% to 20% of it, where a left to right search comes within 10% to 57%, and it is exact on all 35 rows, with no disagreement against the brute force scan. It reads fewer positions than a left to right search on 22 of the 35 rows, by up to 1.57 times at $m = 64$ and 128.

The number of probes is small without any effort to make it small. Nine probes settle `structured` at $m = 256$, against $\log_2 256 = 8$, the number deterministic sampling reaches by analyzing the pattern first. English and uniform take 23 to 27. Nothing here works out a sample, keeps anything between searches, or looks at the pattern before the first read.

Two limits. The logarithmic size in the published method is guaranteed in the worst case, and this is a mean over 64 patterns on five corpora with no guarantee of any kind. And it loses on `periodic16` at pattern lengths 8 through 32, by factors between 0.64 and 0.86. That is the same dependence on position that Sections 4.7 and 4.9.3 fail on, for the same reason.

4.9.8 CHANGES THAT DID NOT HELP Each of these was built, measured on the case it was designed for, and kept in the bench. Two of them are feedback controllers acting on the running average of Section 4.9.3, where the measured variable is a shift: a small integer whose variance is close to its mean.

Resetting the running average when the data changes. Comparing a short window average of the shift with a long window average detects a corpus whose character has changed, and then every offset is reset to its largest possible shift, which puts the probe back at $m - 1$. On the four corpora that do not change, the detector never fires on 15 of 28 rows, and where it fires it moves the result by at most 1%. A fifth corpus made of four parts end to end, English, then C source, then records, then uniform bytes, was built to test it, and on that corpus every method ends within 1% of every other. The reset has no measured benefit and no measured cost.

Probing at the running average's center of mass. Choosing the offset at $\sum_i v_i i / \sum_i v_i$ instead of the offset with the largest v_i loses on every row, by 2% to 42% against the largest and by up to 48% against Horspool. The running average roughly increases with offset, because each entry starts at $a+1$. Its center of mass falls about two thirds of the way along, where the largest possible shift is only middling. A center of mass is a natural location, but the mean of a payoff is not the place where the payoff is largest. The construction that gives Section 2.1 its reference point is the wrong one for choosing among values.

Adding integral and derivative terms to the update. The running average is a first order lag, which is a proportional response with a leak. Adding an integral term at 0.002 and a derivative term at 0.05 gives a mean of 1.005 against the lag alone and is worse on 24 of 35 rows, losing most on the periodic corpus, where the derivative term follows the oscillation. A shift is a small integer

whose variance is close to its mean. The difference between two successive errors is mostly noise, and a term proportional to it mostly adds noise.

Ignoring reads near the average and amplifying the rest. Replacing the lag with a deadband around the running mean shift, applying full weight outside it and relaxing toward the largest possible shift inside it, gives a mean of 1.014 against the lag and is better on 2 of 35 rows.

What the changes have in common. Each changes how the search responds to information it already has. The one change in Section 4.9.3 that did pay, updating every offset from one read instead of only the offset that was read, changes how much information one read yields. Across the seven variants measured here, getting more out of each read gains 13.2% and responding to it more cleverly gains nothing.

4.10 Symbol width and entropy rate

Section 2.1 does not assume that the alphabet can be listed. It does not follow that a symbol is a byte, and nothing in this paper justifies eight bits. A corpus is a string of bits, and where one symbol ends and the next begins is a choice imposed on it. Every H_2 reported before this section belongs to the corpus and that choice together.

Section 2.2 does not mention the width. Correctness holds at every width at once. The width changes cost. A read of w bits costs w bits of reading and yields $H_2(w)$ bits of information. A search spends $w \log_2 N / H_2(w)$. The quantity to compare across widths is $H_2(w)/w$.

Measured over windows starting at every bit offset, since a read can begin anywhere:

corpus	w=1	w=2	w=4	w=8	w=12	w=16
`english`	0.984	0.984	0.969	0.856	0.696	0.600
`structured`	0.967	0.966	0.954	0.850	0.670	0.541
`periodic16`	0.989	0.988	0.944	0.888	0.792	0.726
`uniform`	1.000	1.000	1.000	0.997	0.973	0.855
`flat`	0.678	0.708	0.670	0.375	0.250	0.188

The ratio falls with width on every corpus. Reading English a byte at a time yields 0.856 bits per bit read, against 0.984 a bit at a time. Reading by the byte gives up 15% of the information available in the same number of bits, and a sixteen bit symbol gives up 40%.

The flat corpus is not flat. It is `0x41` repeated, `01000001` has period eight, and a window of six bits or more takes exactly eight distinct values whatever the corpus length. So its collision entropy is exactly $\log_2 8 = 3$ bits. The sweep reports that at widths 6, 8, 12 and 16, and 0.678 bits at width one. The corpus that every other section of this paper reports zero on carries three bits when it is read through a window one bit narrower.

What this does not settle. $H_2(w)/w$ is the right comparison only where reading costs by the bit. Where a read costs the same whatever its width, as it does for a machine word, the quantity to maximize is $H_2(w)$ itself, and that grows steadily with w on every row above. The cost models disagree about which width is best, this paper counts reads under the last of them, and the earlier sections do not say which one they assume.

4.11 Finding a period without a pattern

Every measurement before this one is given a pattern, but the method does not need one. A shift d is the hypothesis that the corpus agrees with itself d positions apart, any two positions read d apart test it, and one disagreement rules it out for good. So the pattern is not one string but every

shift at once, and the reads are shared by all of them: k positions give $k(k-1)/2$ tests. Here 512 reads test 130,816 pairs.

The significance threshold cannot be trusted, and is not used alone. Tests of one shift share positions. They are not independent, and a standard error computed as if they were is too small. With that threshold alone, the uniform corpus, which is SHA-256 output and has nothing to find, reports twelve shifts.

A shuffle of the same bytes, a permutation test [5], settles it. The shuffle keeps the byte histogram exactly. The collision probability is the same, and it destroys every relation between positions, which is all a shift can detect. Whatever the detector reports on the shuffle, it reports on nothing.

corpus	shifts reported	on the shuffle	difference	strongest shift
`periodic16`	92	0	**92**	560
`uniform`	12	16	-4	919
`mixed`	4	6	-2	416
`structured`	1	1	0	984
`english`	0	0	0	1719

One corpus has structure of this kind, the one built with a period. Its strongest shift is 560, and $560 = 35 \times 16$. The detector finds a multiple of the record length without being given the record length or a pattern to look for. The `mixed` corpus peaks at 416, also a multiple of 16, which is its periodic quarter showing through below the threshold.

What it does not find says as much. English prose and C source report nothing beyond their shuffles, and English peaks lower than its own shuffle does. Both clearly have structure, but not agreement with themselves at a fixed distance, the only hypothesis this test checks. Repetition at irregular positions, as in a word or an identifier, needs a different test. The detector finds periods and says nothing about anything else.

4.12 Finding word boundaries

Section 4.11 asks whether a corpus repeats at a fixed distance, and prose does not. A different question applies to it. Whatever else a language is, it is built from units, and something in it marks where one unit ends, a separator or a rule. If nothing does, there are no units. That leaves a trace that the same reads can measure.

If a symbol's positions have no structure, the gaps between its occurrences follow a geometric distribution, and the ratio of the variance of the gaps to their squared mean, the squared coefficient of variation, is close to one. A separator bounds the length of a unit. Its gaps are much more even and the ratio falls. The statistic has no units and no scale. One threshold means the same on any alphabet and any corpus. The shuffle calibrates it as before: it keeps every symbol's count and destroys where the symbols are, and every symbol's gaps then come out geometric.

corpus	symbol found	mean gap	squared coefficient of variation	on the shuffle	ratio
`english`	`0x20`, space	5.58	0.162	0.599	3.70
`periodic16`	`0x0A`, newline	16.00	0.0000	0.653	unbounded
`structured`	`0x3B`, semicolon	59.9	0.353	0.413	1.17
`uniform`	`0x45`	177.6	0.324	0.339	1.05
`mixed`	`0x6B`	134.0	0.814	0.400	0.49

Each corpus that has a boundary marker finds it. English finds the space, at a mean gap of 5.58, which is a word and its separator. The record corpus finds the newline, at exactly 16.00 with a squared coefficient of variation of exactly zero. C source finds the semicolon, which ends a

statement, but only weakly at 1.17, because statement lengths vary much more than word lengths do.

The uniform corpus gives 1.05, which is nothing, and the mixed corpus gives 0.49, which is less regular than its own shuffle. Four kinds of data end to end have no single marker that is regular across all of them, and the statistic shows that, instead of reporting the marker of whichever part is longest.

Nothing here is given a pattern, a dictionary, or an idea of what a unit is. What makes the comparison valid is that both objects are finite and hold the same symbols. The null is constructed instead of assumed.

4.12.1 BOUNDARY SPACING AS A FILTER A known boundary symbol turns a corpus into a sequence of gaps, and a pattern with k boundaries has $k - 1$ gaps between them that every true match must reproduce. That sequence is a filter by itself, before any other symbol is examined.

corpus	m	boundaries in the pattern	candidates	reduction
`english`	16	2.88	37.3	74
`english`	32	5.72	1.75	1572
`english`	64	11.45	1.00	2726
`english`	256	46.0	1.00	2534
`structured`	256	4.27	1.01	923
`periodic16`	64	4.00	252.1	16.0
`periodic16`	256	16.00	240.3	16.0

On English the spacing alone finds the pattern's one position from a pattern length of 64 up, using a symbol that is 17.9% of the corpus and so the worst choice of probe by rarity. The filter gets stronger as the pattern gets longer, because a longer pattern contains more boundaries, and the boundaries are what is being tested.

The record corpus is the counterexample, and it does not change. Its reduction is exactly 16.0 at every pattern length and every number of boundaries, because its newlines are perfectly periodic: the spacing fixes only the alignment modulo 16. A boundary with period T gives at most $\log_2 T$ bits, however many of them a pattern holds.

That reverses Section 4.12. `periodic16` has the most even spacing found anywhere, a squared coefficient of variation of exactly zero, and its boundaries are the least useful for finding a position. English has 0.162, and its boundaries find a single position. A boundary has to be regular enough to be detected and irregular enough to mark a position, and the variation in the spacing is what does the second.

The uniform corpus also comes down to a single candidate, with a marker that is 0.0049 of the corpus. That is the rare symbol probe of Section 4.3 working as usual, not a boundary, and it is what a corpus with no boundaries should give.

4.13 Regularities across languages

Section 4.12 finds a boundary and Section 4.12.1 uses it. Neither says whether what it found belongs to language or to the one English sample it was run on, which the author of this paper wrote. Two regularities are said to hold for every natural language: the frequency of a unit falls roughly as the reciprocal of its rank, and the frequent units are the short ones [35]. Both are explained as effects of communicating with limited effort. Neither should depend on which language it is or when the text was written.

Six public domain texts are taken from Project Gutenberg [21] and read by the whole-search bench. Line endings are first turned into spaces, because a plain text file wrapped at a fixed width has a line break every fifty or so bytes, and the detector would find the publisher's formatting before the language.

text	family	year	boundary	gap	Zipf slope	word length against frequency
Cervantes, *Don Quixote*	Romance	1605	`0x20`	5.13	-1.047	-0.305
King James Bible	Germanic	1611	`0x20`	4.70	-0.925	-0.244
Goethe, *Faust*	Germanic	1808	`0x20`	5.09	-0.723	-0.364
Austen, *Pride and Prejudice*	Germanic	1813	`0x20`	4.94	-0.875	-0.301
Hugo, *Les Misérables*	Romance	1862	`0x20`	5.40	-0.950	-0.321
Kalevala	Uralic	1849	`0x2C`	43.7	-0.563	-0.153

Five of the six find the same boundary symbol without being told there is one, at mean gaps between 4.70 and 5.40. The Zipf slopes are between -0.72 and -1.05, against the -1 the law states, and the correlation between word length and frequency is negative in every row. Cervantes and Hugo are 257 years and two countries apart, and differ by 0.10 in slope and 0.016 in the length correlation.

The Finnish row reflects the kind of text instead of Finnish. Every other text here is prose, and the *Kalevala* is verse in strict trochaic tetrameter, compiled from oral songs. Its lines are more regular than its words. The detector finds the line. Replacing it with Finnish prose settles the question:

text	boundary	gap	Zipf	word length against frequency	distinct words per word
Kivi, *Seitsemän veljestä*, prose, 1870	`0x20`	6.82	-0.828	-0.298	0.37
Kalevala, verse, 1849	`0x2C`	43.7	-0.563	-0.153	0.91
Austen, prose, 1813	`0x20`	4.94	-0.875	-0.301	0.11

Finnish prose finds the space and gives a length correlation of -0.298, against -0.301 for English prose. The meter causes the failure instead of the word structure.

Finnish's long compound words do show, in two places that do not cause the failure. The mean word is 6.82 bytes against 4.94, and distinct forms per word are 0.37 against 0.11, because a language with many grammatical cases gives one root many forms. Neither stops the detector working.

What this does and does not show. Six prose texts, four languages, three families, one of them not Indo-European, from 1605 to 1870, every one in an alphabetic script. This is evidence that the regularities survive changes of language, family, word structure and century within that setting. It is not evidence about logographic scripts or about spoken language, and the one verse text in the set behaves differently enough that the kind of text is a variable this has not controlled for, beyond replacing that text.

4.13.0 ARTIFACTS OF THE FILE FORMAT

Four times in this section the most even spacing in a file belonged to the file, not to what was written in it.

what was found	where
`0x0D`, the line ending of a text wrapped at a fixed width	5 of the 6 texts, before line endings were folded
`Heading`, Project Gutenberg's own markup	Austen's closest company in Section 4.14
`0x2A` at a gap of 2.02 and a squared coefficient of variation of 0.005, a ruled separator	a Greek text
`0x30` at a gap of 816, verse numbering	the same Greek text

The most even thing in a text file is almost never the language, because a language is made by someone composing and a file format is made by a rule. Every result in Sections 4.12 to 4.14

needed a layer of the format removed first, and the ones above were caught by reading the output, not by any check inside the method.

Two conditions now apply to a candidate boundary, and both came out of these failures. A boundary has to have a squared coefficient of variation above 0.05, because Section 4.12.1 shows that a perfectly even boundary only fixes the phase of its own period and carries nothing else of use; decoration and padding do exactly that. And it has to occur at least once in every 64 bytes, because a symbol can be even and still be a page number, which marks the page and not the unit. Both conditions were added while trying to read a text that was failing for a third reason, covered in Section 4.13.05, and both leave every corpus that already worked unchanged.

4.13.05 READING UNICODE AS SYMBOLS Every measurement above counted bytes, and nothing in this paper justified that. Section 4.10 already shows that where one symbol ends is a choice the method has to make, and the byte was chosen here out of habit.

A Greek text exposes the choice. The detector returned `0x68` at a mean gap of 21.24, which is not the word boundary, and the regularities collapsed to a Zipf slope of `-0.049`. The cause is visible in the file without knowing what language it holds. UTF-8 marks its own structure, a lead byte from `0xC2` to `0xDF` opening a two byte sequence and every following byte between `0x80` and `0xBF`, and reading that structure out of the Greek text gives 71.3% two byte sequences, against 99.4% one byte sequences for the English text. A byte level detector on the Greek text was measuring code units, and half a letter is not a symbol.

The alphabet is finite. This is recoverable. Greek has been written with a bounded set of letters in every period it was written in, and the text holds 144 distinct symbols, each of which fits in a byte with room to spare. Re-seating them in order of first appearance, which has nothing to do with any statistic measured, and folding line endings as the byte path already does, the detector returns `U+0020` as the boundary at a mean gap of 4.42, and the text reads:

corpus	Zipf slope	word length against frequency	types at 25k	bits per word
Homer, Iliad (Greek)	-0.878	-0.352	7001	10.318
Austen 1813 (English)	-0.875	-0.285	5690	9.792
Hugo 1862 (French)	-0.950	-0.311	7432	10.324
Goethe 1808 (German)	-0.723	-0.357	8064	10.891
Kivi 1870 (Finnish)	-0.828	-0.313	11841	11.900

Greek sits inside the group on every column, with the second strongest correlation of word length against frequency of any natural text measured here. Nothing in the detector knows Greek, and all it needed was the right symbol width.

Two checks show the re-slicing is not doing the work itself. Applied to the English text, where the byte was already the right width, it moves the Zipf slope from -0.875 to -0.875 and the type count from 5691 to 5690. And the boundary it finds is not planted: seats are assigned by first appearance, the detector ranks candidates by how even their spacing is, and it recovers `U+0020` in all four texts re-sliced this way.

4.13.06 TRANSLATION AGAINST COMPOSITION The 1611 King James and the 1623 Shakespeare are twelve years apart in one language, one translated from Hebrew and Aramaic and one composed. Capped at the same 1048576 symbols, Shakespeare uses fewer tokens, 188474 against 199878, and 92% more distinct types, 25031 against 13058. So the King James has the smallest vocabulary of any prose measured here, and the century is not what put it there.

The correlation of word length against frequency separates the same pair without using vocabulary at all. It is -0.247 for the King James against -0.315 for Shakespeare, and every text composed in its own language is between -0.285 and -0.357. That correlation is a property of composition, because the speaker who shortens a frequent word pays for its length. A translator is not that speaker and chooses for sense, which breaks the link between a word's frequency and its length.

So two measures with nothing in common separate a translation from a composition of the same decade, and a source language whose words carry several senses at once is the mechanism both would be reporting. Neither measure names that mechanism, and neither reads a word.

4.13.07 CONTROLLING FOR GENRE Epic was tested for a register of its own and twice returned nothing, both times for a reason in the instrument.

Reuse of set phrases was the first test, since epic is built from repeated formulas. The Greek Iliad returned 108 formulas per 25000 tokens, against 142 for Shakespeare and 128 for Austen, which is no signature.

Word structure was the first explanation offered, on the reasoning that a Homeric epithet changes its ending with its noun so that no exact match survives, and it is wrong. English barely changes its endings. The test was repeated on two English epics that are each a single work. Milton returns 120 and Pope's Iliad returns 98, both below Austen's 128 on the same prose novel the Greek text was already below. Pope carries the same story as the Greek text, in a language where the stated mechanism cannot operate, and the count goes down instead of up.

So the argument from word endings does not survive, and neither does any claim that epic reuses more set phrases. Either the register has no such signature, or this count does not measure what an oral formula is.

Clustering was the second test, since a large cast of names should cluster. The Iliad returned 4.03, the second lowest of every corpus, while Shakespeare returned 40.06 and the King James 20.77. Those two are collections, 37 plays and 66 books, and a name confined to one member of a collection clusters for that reason. The measure is reporting how many separate works were joined end to end.

So this paper makes no claim about an epic register, and it withdraws the explanation it first gave for not finding one. Three measurements over five texts in three languages, two of them chosen to remove the confound blamed the first time, return nothing that separates epic from prose.

The clustering figures cannot be read on a re-sliced corpus at all, because the reporting path prints the matched unit as bytes, and those bytes are seat indices after Section 4.13.05 re-seats them.

There is a limit behind all three failures that no better instrument reaches. Every measure here reads how a text was produced, and a story someone invented is produced by the same species using the same machinery as a report of something witnessed. The two come out alike because they are alike in the one respect being measured. This is Section 2.3 reached from the side of the corpus: the signature is a necessary condition of human production and never a sufficient one for what the text describes. A text can carry every regularity in Section 4.13 and describe nothing that happened.

4.13.08 DRIFT OVER TIME Drift is a distance between two texts, which needs a comparison the per corpus measures do not make. Two frequency bands are compared by Jaccard overlap. The top 100 words of an English text are almost all function words and move slowly. Ranks 500 to 2000 are mostly content words and follow the subject.

pair	years apart	form	top 100	ranks 500 to 2000
Milton 1667, Pope 1720	53	both epic verse	0.5625	0.2310
King James 1611, Shakespeare 1623	12	scripture, plays	0.5038	0.1596
Pope 1720, Austen 1813	93	epic verse, novel	0.3889	0.1207
King James 1611, Austen 1813	202	scripture, novel	0.3699	0.1054
Kalevala 1849, Kivi 1870	21	Finnish verse, Finnish novel	0.2121	0.0684
Austen 1813, Hugo 1862	control	different languages	0.0204	0.0132

The last row shows the measure behaves, since two languages share almost nothing and the figure goes to the floor. The first two rows show it cannot answer the question. Fifty three years with the form kept fixed keeps more of the shared vocabulary than twelve years across a change of form. The genre term is larger than the time term, and one genre matched pair cannot separate them.

A second objection is stronger than the statistical one. A drift floor is a claim about a population living normal lifespans in one place, with nothing breaking the chain of transmission, and the only genre matched pair available spans the English Civil War, the regicide, the plague of 1665 and the fire of 1666. That interval is the opposite of the condition. 0.5625 is not a floor. Read as an upper bound on how fast the shared vocabulary moves, it says about 72 of the 100 most frequent words survived that half century unchanged.

Measuring the floor needs several pairs matched for form at different separations, drawn from a period with no disaster and no displacement. This corpus holds no such series.

Compared against one 1611 reference, the English texts give 0.5038 at 1623, 0.4286 at 1667, 0.4184 at 1720 and 0.3699 at 1813 in the top 100 band, which falls at every step. The content band reverses once, rising from 0.1596 at 1623 to 0.1632 at 1667, and Milton is retelling the subject of the book being compared against. That reversal comes from the topic, and the date plays no part in it.

Turning the top 100 band into a loss per year gives 0.00171 across 1623 to 1667, 0.00019 across 1667 to 1720, and 0.00052 across 1720 to 1813. The first interval covers the civil war, the regicide, the plague and the fire, and it moves 8.9 times faster than the second, which covers the settled period after the Restoration. A drift rate that responded to disruption would produce that order.

That order is not evidence of one. The 1667 to 1720 interval is the only pair in the series matched for form. The slowest interval is also the one where the largest confound was removed. The disruption term and the genre term take the same value on the same rows, and nothing measured here separates them. Whether a shock produces an oscillation is a further question this corpus cannot reach at all: five samples spread unevenly over 202 years are fewer than two per cycle for any cycle worth proposing. No period is detectable and none is ruled out.

Holding the subject fixed removes the genre term by construction. Three translations of one book were measured against each other. The Douay Rheims of 1609 and the King James of 1611 are two years apart and were made by rival translators from different sources, which prices the translator's choices with the era kept still. The World English Bible is 389 years after the King James.

pair	years apart	top 100	ranks 500 to 2000
Douay 1609, King James 1611	2	0.8519	0.5440
King James 1611, World English 2000	389	0.7857	0.6243

Douay 1609, World English 2000	391	0.7699	0.4684

Every figure here is far above the genre varied pairs above, which confirms the subject was carrying most of that difference. Within these three the era is the smaller term. Two years across a change of translator costs 0.1481 of the top 100 band, and 389 years costs 0.0662. The language moving for four centuries does less to the shared vocabulary than the choice of who is translating.

The content band settles it. The King James and the World English agree better across 389 years, 0.6243, than the two translations two years apart do, 0.5440. The Douay Rheims came through the Latin Vulgate, and the World English descends from the American Standard in the King James line. That column is reporting which textual tradition a translation belongs to.

Three designs have now been tried and each lost to a different confound: pairs matched for form lost to genre, a fixed reference series lost to genre, and a fixed subject lost to the translator's lineage. Every series reachable here varies something that moves vocabulary harder than time does. No drift rate is reported, and whether changes bunch against periods of disruption is not tested. What the last table does support is the stability itself. A top 100 overlap of 0.7857 across 389 years is a shared vocabulary that barely moves, and any bunching would have to be visible against it.

There is a further limit in the measure that no extra corpus removes. Across 389 years of one language it reads 0.7857, and across two languages it reads 0.0204, and nothing measured here falls between those. The instrument has a slow range and a total range with nothing resolved in between.

That gap is where a language replacement sits. A change at the edge of a civilization replaces one language with another instead of changing a rate, and replacement is a discrete event, a way for changes to arrive in a group that does not need the drift rate to vary at all. Measuring it needs a replacement with text on both sides, and the drift rate is the only quantity these measurements compute. For the oldest replacements there is no text on either side and there never will be. Writing is one channel among several, and this paper cannot read the others at all. A gap in the written record across an event is not evidence that continuity failed there.

There is a selection effect in this that no corpus work removes. The events most likely to show changes arriving in a group are the ones that leave nothing to read, and the intervals that can be measured are the ones mild enough that a writing tradition survived them. A corpus of written language is a sample of the surviving cases, and it is biased against the very thing being asked about.

A pattern can be certainly present and its meaning permanently lost. That is Section 2.2 holding while Section 2.3 does not, and the filter of Section 2, run backward on a real object, shows it.

An Attic black-figure amphora signed by Exekias, Achilles and Ajax at a board game (Vatican inv. 16757, 540 to 530 BC; the photograph is by Daderot, released CC0), was read at 160 by 107 gray levels, copied fifteen times, and each copy had 40 percent of its pixels replaced by a value from nowhere. The art example's recovery, told only that the subject repeats, recovered the image by taking the agreed value at each pixel. All 17,120 pixels came back exact, a noise reduction of 100 percent, and the filter flagged 896 of them, 5.23 percent, as a floor it would not certify because its two exact routes disagreed there. Not one flagged pixel was wrong, and not one wrong pixel went unflagged. Figure 1.1 shows the clean image, one damaged copy, the recovery and the map of uncertainty.

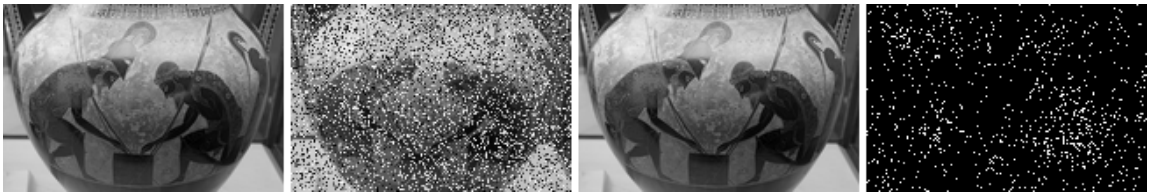


Figure 1.1: A real Indo-European object recovered to the bit. Left to right: the Exekias amphora detail read at 160 by 107 gray levels; one of fifteen copies with 40 percent of its pixels replaced by a value from nowhere; the recovery by agreement at each pixel, exact on all 17,120 pixels; and the map of uncertainty, 896 pixels (5.23 percent) flagged as the floor the filter would not certify. The signal returns exactly and its meaning does not. Photograph by Daderot, CC0.

The filter recovered the object to the bit and recovered nothing of what it meant. Ajax, Achilles, the game between them and the throw Achilles calls out are not in the pixels, and no agreement over copies returns them. The pattern is present and recoverable to the last bit, and its meaning is not the filter's to give. That is the asymmetry this section reports, on a real object instead of a ruin.

None of this makes the regularities fragile, and the reason is the only result in this section that came from a controlled destruction instead of a comparison. Section 4.13.1 permutes the Morse code table, the part a person designed and the part actually sent, and the correlation of word length against frequency returns at -0.550 against -0.652 for the intact table, while the Zipf slope does not move, -1.183 against -1.185. The assignment of codes to letters was destroyed and the measurements reassembled out of the wreckage, because they were never encoded in that assignment. They belong to the distribution of the words being encoded, and they reappear through any re-slicing of the symbols.

So these regularities are not transmitted and do not need a surviving channel between two instances of them. They are properties of whatever produces a message, and they recur wherever that kind of producer recurs. Three language families, two scripts, 265 years and one tapped-out medium give the same measures for that reason, and not because anything passed between them.

The mechanism is that a shorter code for a more frequent symbol lowers the expected cost of sending, which is a result in coding theory and not a fact about people. Any system that pays per unit sent, over enough repetitions, arrives at it. So does a telegraph operator assigning codes by hand, and Section 4.13.1 finds most of Morse's word-length effect surviving the destruction of the table he designed. A language is solved against one set of constraints, a vocal tract of fixed bandwidth, a working memory holding a few items, a listener decoding in real time, and a finite childhood to learn in, and the best answer belongs to that set of constraints and not to any lineage. Nothing inherits the answer. Each language derives it again, because the problem is still there. Losing the content therefore does not remove the regularity: the regularity was never stored, and the conditions that regenerate it outlast every artifact.

One caution belongs here, and it is measured. Distributions of this shape also come from processes doing no optimizing at all, and Section 7.4 runs that counterexample against these same measures. Independent character draws with a separator return a Zipf slope of -0.988, inside the range every natural corpus here occupies. The Zipf slope in this section carries no evidence about production and is withdrawn as such. The correlation of word length against frequency survives,

weakened, at -0.222 for the control against -0.247 to -0.364 for natural text. The boundary regularity against a permutation null survives intact, at 1.00 for the control against 1.79 and above for natural text. That counterexample does not weaken the method in this paper; it strengthens it. A structure produced by nearly any generator is a better thing to rely on than one belonging to a special class of them, and the requirement here was never that the source be a language. What the measures here need is that something regular produced the bytes. The filter of Section 2 needs less, a test of equality between two positions (Section 2.1), and it needs neither alphabet, dimension nor interpretation.

The measures in Section 4.13 are also close to a question that has been asked before, since a Zipf slope and a word-length effect are used to decide whether an undeciphered corpus is a language at all [29]. That literature exists and is contested. It is not cited in full here because it has not been read in full, and Section 7 states what was read and what it cost.

4.13.09 STRUCTURE BY FREQUENCY BAND Section 7.4.1 finds the two halves of the frequency distribution behaving differently, the commoner half carrying even boundaries and the rarer half carrying clustering. Neither measure reads a word. Printing the twelve most frequent words of each corpus shows what fills the half that all the statistics describe without naming:

corpus	twelve most frequent words
Austen 1813 (English)	the to of and her i a in was she that it
King James Bible 1611 (English)	the and of to that in he shall unto for i his
Hugo 1862 (French)	de il la et le l à un les que une d
Goethe 1808 (German)	und ich die der nicht das ein ist zu du in sie
Cervantes 1605 (Spanish)	que de y la a en el no los se con por
Kivi 1870 (Finnish)	ja mutta hän juhani on niin kuin nyt oli ei he hänen
Homer, Iliad (Greek)	και κι του ο το να τον τα τους με τ η

Every commoner half carries words that refer to people. English has `her`, `i`, `she` and `it`, German has `ich`, `du` and `sie`, Finnish has `hän`, `he` and `hänen`, and Greek carries `του`, `τον` and `τους`. Each also carries a conjunction, and each carries a linking verb, a negation, or both. Finnish is the clearest case: it has no articles to fill those positions, and a proper name, `juhani`, takes one of them.

The matching claim about the rarer half is that it carries the actions, and that needs counting. No part of speech tagger is available here. English verb endings stand in for one, and the stand-in is weak in one direction: a gerund used as a noun and an adjective built from a participle both carry these endings without being verbs. Every figure is an upper bound.

corpus	commoner half, percent ending in `-ed` or `-ing`	rarer half
Milton 1667 (English)	10.1	22.3
Shakespeare 1623 (English)	9.7	18.5
Austen 1813 (English)	16.9	23.7
King James Bible 1611 (English)	12.9	15.4
random typing, English letter weights	0.0	0.0

The rarer half carries more of it in all four, by 1.19 to 2.21 times, and the memoryless control produces none. The enrichment is not an artifact of counting endings. The King James is the weakest row, and its own word forms explain that: it conjugates as `saith`, `cometh` and `spake`, which this stand-in does not count at all. Its verbs are undercounted by an amount not estimated here.

So the commoner half holds the machinery for referring to people and saying something about them, and the rarer half holds more of what is being done. That describes the two halves. The description reaches nothing about what any text means, and the statistics in Section 7.4 that separate a corpus from a shuffle of itself do not know that either half exists.

Scoring individual words for burstiness against the same shuffle answers two questions, and only the second comes out.

The first was whether words for things people do constantly, instead of things a passage is about, spread more evenly than the rest. Section 7.4.4 predicts they should. A supplied English set of them gives 0.843 against a corpus average of 0.828 for Austen, 0.521 against 0.509 for the King James, and 0.809 against 0.796 for Milton. The direction is the predicted one every time, and the size is not worth reporting: between 7 and 20 words cleared the occurrence floor, and the prediction is not established.

The corpus averages themselves separate, which the first question was not asking about.

corpus	word burstiness	what the corpus is
random typing, geometric letter weights	1.010	a memoryless process
Homer, Iliad (Greek)	0.833	one poem
Austen 1813 (English)	0.828	one novel
Milton 1667 (English)	0.796	one poem
Cervantes 1605 (Spanish)	0.780	one novel
Pope's Iliad 1720 (English)	0.780	one poem
Kivi 1870 (Finnish)	0.770	one novel
Hugo 1862 (French)	0.758	one novel
Shakespeare 1623 (English)	0.667	37 plays
C source	0.533	40 files
King James Bible 1611 (English)	0.509	66 books

Single works occupy 0.758 to 0.833 over four languages and three centuries, collections occupy 0.509 to 0.667, and the memoryless control sits at 1.010. The bands do not overlap. The C sources fall with the collections, and forty files each about a different module should do that.

Hugo is the lowest of the single works, and that supports the reading instead of straining it. *Les Misérables* runs to five volumes and turns aside into Waterloo, the sewers of Paris and the history of a convent. It is the most varied single work measured here in what it is about, and it sits nearest the collection band.

Every corpus in that table arrived already being one thing or the other. The reading rests on labels applied by hand and could be tracking whatever those labels happen to go with. Two manipulations settle it. Joining English single works into one corpus, with the total length kept near 900000 characters so the change cannot be a length effect, gives 0.828 for one work, 0.685 for two and 0.626 for three. Cutting the 66 book collection into equal pieces moves the other way:

pieces	books per piece	mean	highest piece
1	66	0.509	
8	8.2	0.612	0.700
16	4.1	0.648	0.740
32	2.1	0.681	0.835
64	1.0	0.708	0.847

The climb continues at every cut, and at one book per piece the highest pieces reach 0.847, which is inside the band the single works occupy. The mean stays at 0.708 for two reasons that are properties of the cutting: the pieces are equal in length and therefore do not line up with book boundaries, most of them straddling two books; and a short piece has fewer words clearing the occurrence floor.

So the quantity responds to subjects being added and removed while length is kept fixed, which is a stronger statement than the table of labeled corpora supports on its own.

The measure is not a count on its own. Two English works joined give 0.685, close to the 0.667 of a corpus of 37 plays, because a novel and an epic poem stand further apart than two plays of one period do. The measure reads how far the vocabulary spreads across subjects, with the number of subjects and the distance between them both contributing and neither separated here.

That is a property of what a text is about, recovered without reading any of it, and it is the only measurement in this paper that orders corpora on a scale with separated bands. It still names no subject and identifies no meaning. It counts how far the vocabulary spreads.

4.13.1 STAYING THE SAME UNDER RE-ENCODING

Every row above is alphabetic, and a regularity common to all of them could belong to that way of writing and not to language. Re-encoding one of them settles it, because a re-encoding changes only the writing and holds the meaning fixed. Morse code carries the same words over two marks and two silences.

	boundary	gap	squared coefficient of variation	Zipf	word length against frequency
Austen 1813, as written	`0x20`	4.94	0.374	-0.875	-0.301
Austen 1813, in Morse	`0x20`	4.59	0.217	-1.185	-0.652

Given four distinct marks and no letters, the detector finds a boundary and both regularities are present. The correlation of word length against frequency is more than twice as strong, which looked at first like the one result here that was designed, not discovered, since Morse gives its shortest codes to the most frequent letters. Section 4.13.2 tests that, and it is mostly not so.

The two rows are not at the same level and should not be read as a pair. English splits at words. Morse splits at the gap between letters, because within a word that gap is more even than the gap between words. The unit counts say so: 1187 distinct units is far more than the 36 codes Morse has, and the encoder writes the word separator with no space beside it. The last code of one word, the separator, and the first code of the next form one unit. Thirty six codes plus up to 1296 such pairs brackets the 1187 seen. The transcription invented a vocabulary of boundary spanning pairs that is in neither language.

What the row does establish is narrow and is the part worth having. Neither regularity is a property of alphabetic writing. Both survive into a four symbol tapped-out form of the same text.

4.13.2 PERMUTING THE CODE TABLE

Morse had regional variants that gave different codes to the same letters, and a code table is a choice, not part of what is being said. Permuting it holds the message, the language and the set of code lengths all fixed and changes only which letter got which code. That separates a property of the message from a property of the encoding, and the two regularities should come apart under it: how often a word recurs cannot depend on how its letters are defined, and how long a code is was somebody's decision.

	Zipf	word length against frequency	encoded bytes
Morse, the real table	-1.185	-0.652	2,001,933
Morse, the same codes permuted	-1.183	-0.550	2,580,891

The Zipf slope is unchanged to two parts in a thousand. A property of the message has to do that under a relabeling of its symbols.

The correlation of word length against frequency is nearly unchanged, which refutes the reading offered in Section 4.13.1. A permuted table keeps a correlation of -0.550. At most 0.10 of the -0.652 can be credited to Morse giving short codes to frequent letters. The rest is structural: short strings recur more often because there are fewer short strings, and that holds for any table whatever. The stronger correlation in the Morse rows than in the English row is mostly an artifact of how the unit lengths are distributed instead of evidence of design.

What the design does buy is the length. The real table encodes the same text in 22.4% fewer bytes than a permutation of its own codes. That is Morse's engineering, and it shows up in the cost of sending and not in the correlation it was credited with.

So the two are not the same kind of claim. The Zipf slope stays the same under re-encoding and is about the message. The correlation of word length against frequency is also largely unchanged. That makes it mostly a fact about how many short strings exist and only slightly a fact about anyone's choices.

4.13.3 VARIATION WITHIN A LANGUAGE Every result above compares one text in one language against another text in another, and a difference could belong to the language or to the text. Separating those needs repetition inside a language, and the sections above have twelve English texts and one or two of everything else.

Four texts were taken in each of seven alphabetic languages from Project Gutenberg's own index for each, which is not a hand chosen selection, and four Chinese novels beside them. Everything is measured by character and not by byte. That is required for the Chinese to be comparable at all, since one of those novels carries 3164 distinct characters, which no byte seating reaches, and it is also the width at which a Chinese symbol is a whole morpheme and not part of one.

The question is then a question about variance. If a quantity belongs to a language, its spread within one language is small against its spread between languages, and an analysis of variance reports that.

quantity	spread within	spread between	F	p
mean distance between boundaries	0.3327	0.6980	13.21	3.5×10^{-6}
collision entropy	0.0430	0.0746	9.02	6.2×10^{-5}
rarer half against a permutation null	0.0731	0.0342	0.66	0.68

The first two belong to a language. How long its words are and what its set of characters looks like separate the seven at p below 10^{-4}. Those are constants of a language, and the earlier readings of them are readings of the language.

The third does not. Its spread between languages, 0.0342, is smaller than its spread within one, 0.0731. Two novels in one language differ on it by more than two languages do. That quantity carries no information about which language produced the text.

Chinese decides it. Measured in within language standard deviations from the mean of the seven, it stands 66.0 away on collision entropy and 22.1 away on the mean distance between boundaries, and 0.5 away on the rarer half. A writing system that encodes morphemes instead of sounds, with thirty times the set of symbols, moves every structural quantity by tens of deviations and leaves the spread measure where it was.

The results in Sections 4.13.05 and 4.13.1 hold over seven language families and two scripts for that reason. The measure was never reading which language it was given. A change of language had nothing in it to disturb. That also bounds what those results can mean: a quantity that cannot tell Finnish from Chinese cannot be evidence about any particular language, and Section 7.4 records separately that it is not evidence about language at all.

This conclusion is not new, and the credit is not this paper's. Montemurro and Zanette [14, 34] measure the entropy a text loses when its words are shuffled, the same shuffle used here, over 7077 texts in eight corpora spanning the Indo-European, Finno-Ugric, Austronesian, Afroasiatic and Sino-Tibetan families and the Sumerian isolate. They report the quantity staying near 3.3 bits per word across all of them, with a relative variability of 0.07 against 0.23 for the entropy itself, and they name it a statistical universal of language. Their sample is two hundred times the size of this one and reaches Old Egyptian and Sumerian, which this one does not.

The statistic here is not theirs, since they measure a reduction in Shannon entropy in bits per word and this measures the spread of the gaps over the rarer half of the symbols, and the agreement of two unlike statistics is worth something. The finding is theirs.

What this paper adds has to be stated carefully, because two different instruments appear in it and earlier drafts ran them together. The spread measure used here has been applied to text, to source code and to recorded sound, which are unlike media, and it separates ordered data from unordered data in each. It has not been applied to pictures or to the crystal lattices, which were measured by the shift detector instead, a separate instrument with a separate shuffle.

Even within the media it does cover, the readings are not directly comparable. A waveform has a smoothness that text does not, the symbol width differs between them, and Section 4.10 shows that the width decides what is visible. So the claim that holds is that one measure runs on unlike media and separates in each of them, not that it returns comparable values across them.

Four texts in a language is a small sample, and all of them come from one publisher. The figures for any single language here carry the choices of that source with them. The variance analysis needs repetition, which it has, more than breadth.

4.14 The variety of a word's company

Frequency cannot separate two kinds of common word. In an English bible `the` and `God` are both frequent, one because it attaches to anything and the other because the book is about it. The company separates them: a word doing grammatical work has nearly as many distinct followers as occurrences, and a word belonging to a narrow subject has far fewer.

The lowest variety among words seen at least 24 times, over the first 25000 words:

King James Bible	seen	followers	variety	Austen	seen	followers	variety
`years,`	32	1	0.031	`Heading`	29	1	0.034
`king`	25	1	0.040	`Mrs.`	73	17	0.233
`said`	133	11	0.083	`Mr.`	178	46	0.258
`sons`	53	7	0.132	`Miss`	119	32	0.269
`And`	788	120	0.152	`I`	354	107	0.302

Austen's three lowest, after one artifact, are the three honorifics. A title in a Regency novel comes before a small fixed set of surnames, and that makes the social system the book runs on visible as a statistic.

The bible's are its register and its set phrases. `And` occurring 788 times at variety 0.152 is very low for a conjunction, and it is the strung-together style the translation is known for. `years,` and `king` each have exactly one follower across every occurrence, and those are the genealogy and the formula for a reign.

Nothing here knows English, knows what a name or a title is, or has a dictionary of any kind. **Two limits.** `Heading` is Project Gutenberg's markup, not Austen, the same kind of artifact as the line wrapping in Section 4.13. And narrow company marks set-piece context in general, which returns stock phrases such as `said unto` alongside words that carry cultural weight, and does not tell the two apart. Cultural weight is one of several things that produce this signature.

4.14.1 BURSTINESS AND GRAMMATICAL WORDS

Narrow company catches grammatical words along with subject words, and a grammatical word is the wrong answer twice over: it is needed everywhere. It says nothing about what a text is about, and its form varies between languages while its job does not. Where a word falls separates the two. A word doing grammatical

work is spread evenly. The gaps between its occurrences are close to geometric and their squared coefficient of variation is near one. A word belonging to a subject arrives in bursts where that subject is discussed, and its value runs far above one. That is Section 4.12's statistic read at the other end.

King James Bible	seen	burst	Cervantes	seen	burst	Austen	seen	burst
`years,`	32	20.8	`Donde`	48	18.5	`you`	205	4.3
`begat`	67	20.0	`Sancho`	52	9.7	`I`	354	3.6
`lived`	34	13.6	`aventura`	37	8.0	`Bingley,`	26	3.4
`king`	25	9.5	`trata`	25	7.8	`had`	179	3.0
`waters`	28	9.4	`historia`	25	4.0	`Miss`	119	2.8
`Esau`	28	8.3	`dijo`	51	3.9	`he`	195	2.5

The grammatical words are gone. Neither `the`, `and`, `of` nor `And` appears in any of the three lists, though each is among the most frequent words in its text. Being spread evenly puts them at the floor of this statistic, and they are discarded without a stopword list, a grammar or a dictionary.

What comes back is the begetting and living of Genesis, Sancho and his adventures, and the pronouns of a novel carried by dialogue. Three languages and three centuries through one statistic.

The size of the numbers carries something too. The bible bursts between 6.6 and 20.8 and the novel only between 2.5 and 4.3. A collection of books written centuries apart has vocabulary that clusters hard by section, and a single novel is even enough that little clusters at all. The size of the statistic reports how uniform a text is.

5. What the results mean

5.1 The algebra of the filter

Take the cost away and what remains is small. The state is one bit per alignment, alive or dead. A read of one position selects a mask over the pattern's positions and clears the alignments whose position holds something else, and the candidates are the complement. That bookkeeping contains no comparison that steers the flow of control, no position computed from a value just read, no order on positions, no metric, and no dimension. A search that chooses its reads, as Section 4.9.7 and the engine's planning pass do, adds comparisons of candidate counts that decide which position is read next and whether another is read, and the count after the full comparison is the same whatever they choose (Section 2.7).

Section 2.2 is why the pieces combine in any order: ruling an alignment out depends only on the position that produced it, and a set of reads contributes a union, which does not remember the order it was built in. Everything else follows mechanically. Reads can be issued together because none waits on another. The bookkeeping is a bitwise OR. The set of candidates is a complement. The mask is m bits wide whatever the alphabet holds and however many dimensions the positions have. Neither quantity appears in the state and neither needs a bound. The vector of alignments grows with the data, and the space bounds proved for streaming matchers do not describe it (Section 7.5).

Entropy is nowhere in that description. It enters one line later, when the question becomes how many reads are needed, and it never returns.

5.2 Collision entropy as the cost parameter

Section 4.0 lists eight quantities that are each a function of 2^{-H_2} and the two sizes. They were derived at different times for different reasons, and several were measured before anyone noticed they were the same parameter: the candidate rate, the expected distance to the first mismatch, the estimator's bias correction, the fraction a left to right search uses, the offset where the shift stops growing, the number of probes before the information runs out, the probe spacing for any order, and the candidate count in d dimensions.

The last of these makes the collection meaningful and not a coincidence. The same expression holds from a line to an eight dimensional hypercube and over an alphabet of complex numbers with irrational parts, which is where the parameter shows it is not standing in for the geometry or for what a symbol happens to be.

A single number that sets a cost across every dimension and every alphabet is qualified in three ways, and the next three sections state them.

5.3 Dependence on the symbol width

There is no collision entropy of a corpus. There is a collision entropy of a corpus read w bits at a time, and Section 4.10 measures $H_2(w)/w$ falling from 0.984 to 0.600 on English between one bit and sixteen. Every figure in this paper uses $w = 8$, because a byte is the unit the code happened to be written in.

The clearest sign that the choice is doing work is the corpus this paper calls flat. It is `0x41` repeated, its bit pattern has period eight. A window of six bits or more takes exactly eight values and carries $\log_2 8 = 3$ bits. Five separate instruments report zero on it, in Sections 4.3, 4.4, 4.4.1 and 4.5.1, and their agreement looked like independent confirmation. All five read bytes. The object carries three bits one bit narrower, and the five agreements are one assumption made five times.

5.4 Information beyond the collision probability

Two results in this paper depend on more of the distribution than its collision probability, and neither is predicted by it.

Section 4.3.1 gives what a probe rule can be worth as the rate of an uninformed probe over the expected minimum of m draws weighted by size, and that ratio is exactly one when the probabilities are equal on the symbols that occur. It is a property of the order statistics. Entropy puts the corpora in the wrong order: `structured` has the lowest collision entropy of the four and the highest ceiling. Section 4.8 then finds a mismatched reference costing 25 to 30 times, which no single-symbol distribution predicts.

5.5 Where independence fails

Every expression in Section 5.2 treats probes as independent. Four measurements say they are not.

Multiplying the rates promises too much, by up to four orders of magnitude at six probes on structured data, always in the direction that leaves too little room. Correlation survives to a distance of seven, and a distance equal to a record period is the strongest effect in this paper at

$z = 12.3$. The count of probes to use up the information predicts 2.98 and measures 6 at $m = 16$, where sixteen offsets cannot be spread far enough apart. And the running average of the shift, which wins 13.2% elsewhere, is beaten on periodic data alone, because crediting every offset from one read assumes the distribution of symbols does not depend on position.

Those are four faces of one thing. The parameter describes a single position, and each failure is a statement about the joint distribution. The boundary is not an accident of these corpora. It is where a single position's distribution stops being able to say anything.

5.6 Results that carry over

The failure of the product rule carries over. Any structure that stacks cheap filters and sizes itself by multiplying their rates inherits it on structured data, in the same direction, and leaves too little room. That carryover is a conjecture: the failure is measured only on probe sequences, here and in the engine workbook, and on no other filter structure.

The estimator is a subsampled U-statistic and loses to a histogram estimator wherever one can be built. Its interest is that it can be computed when one cannot: no bins, no codebook, constant state, and free inside a search that was happening anyway.

And the shape of the argument carries whether the algorithm does. Correctness was separated from cost at Section 2.2 and never rejoined it. Every claim in this paper that survived is one where that separation was kept, and every claim that fell is one where a cost measurement was allowed to stand in for a property of the thing.

6. Limits and what holds

Everything below is finite, and the point of this section is to say what finite means as a number.

6.1 Finite data

Corpora are 1408 to 2048 bytes in the sift bench, 3375 to 4089 positions in the lattice bench, and 256 to 16384 symbols in the estimator bench. Pattern lengths reach 2048 and probe counts reach six. Section 2.2 holds for unbounded sets and the measurements do not. The results that hold unchanged are evidence that the code agrees with Section 2.2 over the ranges named, and are not evidence for Section 2.2 itself, which needs none.

6.2 Finite alphabets

Symbols are drawn from an alphabet of at most 256 values in every bench, and the complex data uses two. The claim that the method tolerates an alphabet that cannot be listed is supported here by the core never reading a value and by the engine's only access to a symbol being equality. No measurement supports it, because a measurement on an alphabet that cannot be listed cannot be made. A growing alphabet is measured instead: from 2^8 to 2^{24} symbols, the memory of the engine's equality route stays at 131057 bytes while a table indexed by symbol grows with the alphabet. Past 256 classes the engine's rank count is an upper bound (Section 2.7).

6.3 Finite precision

The complex data's symbols are square roots of primes stored in `double`. The stored values are rational approximations of irrationals. Equality is decided over the stored bytes. The approximation affects which symbols count as distinct and never whether the comparison is correct.

6.4 Sensitivity to the entropy estimate

Sections 4.6.1, 4.9.2 and 4.9.4 derive constants that take H_2 as an input. All three were computed here from a full histogram over the corpus, which assumes the alphabet can be listed. Section 2.1 refuses that assumption, and Section 4.5 measures what the alternative costs: estimating H_2 from candidate counts carries a root mean square error between 0.02 and 0.60 bits, depending on the number of probes.

Two of the three constants grow exponentially in H_2, so that error does not stay small:

constant	sensitivity	at 0.25 bits	at 0.60 bits
shift stops growing at $3 \cdot 2^{H_2}$	$\mathrm{d}r/r = \ln 2 \cdot \mathrm{d}H_2$	17%	51%
fraction used, through $m\,2^{-H_2}$	the same exponential	17%	51%
probes to use up the information, $\log_2 N/H_2$	$\mathrm{d}n/n = -\mathrm{d}H_2/H_2$	6.8%	16%

At 16 probes the distance where the shift stops growing is known to about half its value. For `periodic16` that moves the crossing from 73 to somewhere between 36 and 110, a range spanning two of the pattern lengths tested. Which side of that crossing a given search is on is not always decidable from a cheap estimate. The probe count to use up the information is the mildest of the three and stays inside 16%.

None of this reaches Sections 2.2, 2.3 or 4.1 to 4.2. No match lost, no converse, and every unchanging row are all indifferent to whether the distribution is known. That made them deductive in the first place.

6.5 Pitfalls in measuring the method

Each of the following produces a result that looks correct and is not.

- **A pattern drawn from the data matches itself.** Every predicted candidate rate has to include that guaranteed match. Left out, it inflates every ratio of observed to predicted; in the dimension sweep it drifts the ratio to 1.22 at the smallest data. This is the most common of the pitfalls, and every model of the rate has to be told about it separately.
- **A self similar corpus manufactures trends.** A corpus made by repeating a unit produces a clean trend of correlation growing with pattern length, and the trend reverses once the corpus stops repeating. The corpora are therefore cut off at their length and never repeated (Section 3.3).
- **A control is read by its z before its ratio.** A control ratio of 1.24 is noise when its z says so, and a uniform control whose counts are small does not support a ratio at all.
- **A table matched to the corpus does not win.** Section 4.8 refutes it.

- **The uniform control depends on its generator.** Under SHA-256 in counter mode the uniform skip figures are 271.9, 273.6 and 281.3; under a 64-bit xorshift they are 261.1, 262.7 and 261.1. Every uniform figure here is from SHA-256 in counter mode.
- **Entropy is not what decides a probe rule's worth.** The order of the corpora contradicts it, and Section 4.3.1 gives what does.

6.6 What was not measured

Any cost model that tests many positions at once. Every figure in Section 4.9 counts reads one at a time, and a shift is serial by nature, because the next position cannot be computed until the current one has been read. Under a model that tests many positions per word operation, the shift advantage that decides Section 4.9.1 may not be available at all, and ruling out would be the only lever left. The engine's one wide path is the planner's candidate scan (Section 2.7), which is not this arrangement. That measurement would still decide whether the arrangement in Section 4.9.3 belongs in the library, and it has not been made.

The estimator against published estimators other than the two in Section 4.5. The method on data whose alphabet genuinely cannot be listed. Data correlated in space over more than one dimension, on which a center of mass probe rule would need to be tested, since on independent data every point of a pattern is interchangeable and such a rule correctly shows nothing. Any timing figure: what a SWAR word costs is a question for each processor, and no single machine's number answers it.

7. Earlier work

Two sources were read in full. Both were reached because a search suggested they were close, and a search result is not a reading.

The packed-pair prefilter of the Rust `memchr` library chooses two byte positions by a frequency rank and uses the pair as a prefilter. The only condition it places on where the two sit is that their positions differ, and nothing in it treats their distance as a variable or their rates as independent. This describes the packed-pair prefilter alone. The library's substring search, rare byte prefilter and vector backends were not examined, and no claim is made about them.

The arXiv preprint at `2601.03271` chooses a single probe, $a = \arg\min_i \mathrm{freq}(P[i])$, and reports character comparisons on the *Divina Commedia*. Having one probe, it contains no treatment of distance, independence or joint probability, and uses no periodic or fixed width data.

Neither document contains the separation made here. That is a claim about two documents, not about the literature, and it is the strongest form the evidence supports.

7.1 Order-free filtering and deterministic sampling

Section 2.2 makes ruling an alignment out depend only on the position that produced it. The set of alignments a group of reads rules out is a union and does not depend on the order they were read in. That observation is the dueling paradigm, introduced by Vishkin, and its consequences were worked out at the time. Read from Neuburger's survey, sections 1.3 and 1.4.

A duel eliminates candidates pairwise from single text symbols with no traversal order, and the survey states the setting directly: "This alphabet independent method is used for fast pattern matching in a parallel setting." Alphabet independence, which Section 2.1 assumes, is a stated property of the 1990 method.

Deterministic sampling then chooses probe positions from the pattern before any text is read. "The deterministic sample is a set A of positions of the pattern and a number x, such that if the positions match a candidate i in the text, then all other candidates in the interval $[i-x+1, i+\frac{m}{2}-x]$ are eliminated." Its sample size is logarithmic in the pattern length, which is a stronger result than any fixed stride over the text.

Rare symbol selection appears inside that construction. The pattern is stacked $\frac{m}{2}$ times, a witness column is chosen, and the rule is to "select the less frequent of the two symbols, which can occur in at most half of the copies," repeated at most $\log \frac{m}{2}$ times. Choosing by rarity to halve a candidate set inside an order free elimination is the 1991 method.

Periodicity is a stated precondition there, with a published repair. The guarantee holds "for a non-periodic pattern," and a periodic pattern is reduced by taking its prefix of size $2\pi - 1$, which is not periodic, then linking neighboring occurrences. The failure recorded in Section 4.9.3, worst at pattern lengths equal to the record period and twice it, is that precondition being broken, and the repair for it predates this paper.

What that leaves, stated more carefully than a first reading of it allowed. The earlier work above is combinatorial and worst case. It chooses a rarer symbol without pricing what rarity is worth, and it carries no distribution over the alphabet.

The earlier work also carries a precondition that the method in Section 2 does not. The deterministic sample is built from the pattern before any text is read: the pattern is stacked, a witness column is chosen, the less frequent of two symbols is taken, and the procedure repeats over the pattern's own overlap with itself. That is a preprocessing pass that needs the pattern in hand and can be analyzed in advance, and it buys a worst case guarantee at $\log m$ probes.

An adaptive search skips that pass. It has no set of probes until answers start arriving, and no guarantee either, and Section 4.9.6 measures what that costs: probing without confirming is wrong on a third to almost all of the searches at any spacing above one. What it has instead is that the error goes in one direction only. A miss is impossible under Section 2.2. More probing can only reduce the error and never introduce one, and accuracy rises with effort, with no preprocessing and no limit on how far it can be taken.

Section 4.9.6 also measures why the two cannot be traded freely. The safe spacing is a property of the pattern, and a per pattern preprocessing pass would find it. A spacing tuned on one pattern is wrong on 0 to 62 of 63 others. A fixed set of probes needs the per pattern analysis, and skipping the analysis while keeping a fixed set of probes is the arrangement that fails.

7.2 Average-case string matching

Read from Tsai, *Average Case Analysis of the Boyer-Moore Algorithm* [31], pages 1 to 4 and 14 to 16 of 18. Sections 4 and 5, which carry the Markov chain derivation, were not read.

The quantity that literature computes is C_n, the number of character comparisons, and the constant is $\mu = \lim_{n\to\infty} \mathbb{E}(C_n/n)$. Baeza-Yates and colleagues "applied an analytic approach to the average-case analysis of the BMH algorithm. Under the assumption that the text is independent

and identically distributed (iid), they derived an exact expression for the linearity constant". Mahmoud, Smythe and Régnier established asymptotic normality of C_n, Smythe extended the text model to Markov, and Tsai adds a Berry-Esseen bound.

The parameter is the size of the alphabet. The text is written over "a set of q characters", every numeric result is computed at $q = 2$, 4 and 8, the error term is $(m-K-1)/q^K$, and the experiment uses text "built randomly". Under a uniform alphabet $\sum_\sigma p_\sigma^2 = 1/q$ exactly. $2^{H_2} = 1/\sum p^2$ is the effective size of the alphabet, and the collision probability used throughout this paper is that q carried to the non-uniform case. The transition probabilities in their framework are written for a general distribution. The generalization is available there and is not taken. This paper does not claim a new quantity; it claims the classical one at a weaker assumption.

A comparison count cannot see information a search declined to use. Section 4.9.4 prices what a left to right shift leaves unused, between 3% and 99% depending on the pattern length. That quantity is invisible to C_n, because a comparison count records what the algorithm did, and information the search never acted on is something it never did. The gap is not a defect in thirty years of analysis. It is outside what the measure measures, and it becomes visible only once the order of reads is treated as a free variable, which Section 7.1 credits to Vishkin.

7.3 Thermodynamic bounds on information

Sections 4.3 and 4.8 reach for a physical analogy: a reference distribution is worth how far the data is from it, and at equilibrium there is no gradient and nothing to extract. Read from Parrondo, Horowitz and Sagawa, *Thermodynamics of information* [18].

The results are sharper than the analogy. For a system in a statistical state ρ with Hamiltonian $\mathscr{H}_0$, the non-equilibrium free energy is $\mathscr{F}(\rho;\mathscr{H}_0) \equiv \langle\mathscr{H}_0\rangle_\rho - TS(\rho)$, and Box 1 states that "$-W_{\text{diss}}$ is the maximum work that can be extracted from the non-equilibrium state ρ", with $\mathscr{F}(\rho;\mathscr{H}_0) \geq \mathscr{F}(\rho_0)$ for any ρ. The second law for isothermal processes between non-equilibrium states is $T\Delta S_{\text{tot}} = W_{\text{diss}} \equiv W - \Delta\mathscr{F} \geq 0$.

For measurement the bound is on mutual information, not on a divergence from an arbitrary reference: $\Delta\mathscr{F}_{\text{meas}} = kTI(X;M)$, and for a cyclic feedback process $W \geq -kTI(X(t_{\text{ms}});M)$. The extractable work is "proportional to the information acquired in the measurement". The Szilárd engine saturates it because its outcome is left or right with equal probability, "giving $H(M) = \ln 2$".

Where this paper departs from the analogy. The equilibrium reading holds: data with no structure offers nothing, and Sections 4.3 and 4.8 measure exactly that in the rows that refuse to separate. The reference distribution reading does not survive Section 4.3.1. What a probe rule can be worth here is the rate of an uninformed probe over the expected minimum of m draws weighted by size, which equals one when the distribution is uniform on the symbols that occur. That quantity is not a relative entropy between the data and the table, and the order of the corpora shows entropy does not decide it. The physics is background, and no result in this paper rests on it.

7.4 Identifying language by statistics

Section 4.13 measures a Zipf slope, a correlation of word length against frequency, and a boundary regularity on ten corpora, and reads their agreement as a property of how people produce language. Exactly that inference has been argued over in print. Four items were read in full: Sproat,

Ancient Symbols, Computational Linguistics, and the Reviewing Practices of the General Science Journals [29], and in the same volume the replies from Rao and colleagues, from Lee and colleagues, and Sproat's answer to both.

Sproat's case is that these statistics do not separate language from anything else, and he demonstrates it instead of asserting it. Applying the published classifier of Lee and colleagues to Mesopotamian deity symbols from kudurru boundary stones, a system known to be non-linguistic, returns $C_r = 8.0$ and $U_r = 1.55$ and classifies it as writing. Applying it to 75 "texts" produced by successive tosses of seven six-sided dice returns $C_r = 12.64$ and $U_r = 1.18$, which the same decision tree reports as a syllabic writing system. For the conditional entropy measure he notes that a memoryless process with a Zipfian and non-equiprobable unigram distribution, carrying conditional independence between symbols, reproduces the published curves. His conclusion is that neither result distinguishes "structure that derives from linguistic constraints from structure that derives from some other kind of constraints."

Rao and colleagues answer that sufficiency was never claimed. Their framework is inductive, estimating a posterior over a hypothesis that a corpus is a language from several properties at once, of which entropy is one, and they extend the measurement to block entropies up to $N = 6$, where they report the artificial counterexamples failing to track. They also supply non-linguistic controls that are not artificial, namely DNA, protein, Fortran and music, which occupy entropy ranges away from the linguistic ones. Their own description of the Zipf-Mandelbrot property is that it is "often considered a necessary (though not sufficient) condition for language": the asymmetry between Section 2.2 and Section 2.3 reached from a different direction and by different people.

What this costs Section 4.13, measured and not conceded in the abstract. Sproat's class of counterexamples was built and run through the same code that produced every other row. Characters are drawn independently and a separator is dropped in at a fixed rate.

A control with an alphabet of 26 and a separator rate of 0.18 takes three of its four numbers from English, and a result from it that looks like English establishes nothing. The sweep below varies the alphabet from 8 to 64 symbols, the separator rate from 0.06 to 0.35, and the letter distribution across uniform, two geometric falls and the English weights, with only the last row carrying any value taken from a language.

measure	ten memoryless arms	ten natural corpora	separates
Zipf slope	-0.978 to -1.332	-0.723 to -1.185	no
word length against frequency	-0.124 to -0.401	-0.247 to -0.364	no
types at 25k	5824 to 22160	4171 to 11854	no
spread against a permutation null	0.99 to 1.01	1.79 to 4.82	yes

Three of the four measures fail. The Zipf slope from a geometric distribution carrying no language at all is -0.996, against -0.988 for the English weighted arm. The seeding was never what produced a natural looking value, and being non-uniform is enough on its own. This is the distinction between random and random-and-equiprobable that Sproat says the original work confused, and the uniform arms do sit outside the natural range at -1.224 and -1.332. The correlation of word length against frequency does something worse than fail to separate: an arm over 8 symbols with a separator rate of 0.35 reaches -0.401, stronger than any natural text measured here. The type count overlaps at three arms.

Section 4.13 therefore may not read a Zipf slope, a correlation of word length against frequency, or a type count as evidence of anything about production, and those claims are withdrawn. What remains of that section is the boundary regularity.

That one separates completely, and across the whole sweep. Every memoryless arm scores between 0.99 and 1.01, a corpus you cannot tell from its own shuffle, the figure a process with no memory has to give whatever its alphabet size or separator rate. Every natural corpus scores at or above 1.79. This is the measure Sproat grants, writing of the shuffling method that it "seems to make sense as a way to spot memoryless non-equiprobable processes masquerading as structured systems", and his stated limit on it applies here unchanged: a non-linguistic system that has structure passes it as well. So the surviving measure shows that a corpus is not memoryless. It does not show that a corpus is a language, and this paper does not claim it does.

The surviving measure is also the one Section 7.3 can price. A corpus equal to its own shuffle carries no gradient and offers nothing to extract. The equilibrium reading holds exactly here, and the ratio above one is a distance from that equilibrium.

What a cipher does to the surviving measure. Sproat's stated limit is that a structured non-linguistic system passes this test as well. This section asks what it takes to make a text stop passing it. One English corpus was put through transforms that keep every symbol in the same range of seats. The same measurement path reads all of them.

transform	key length	ratio	mean gap
none		2.91	5.29
one fixed permutation of the seats	1 permutation	2.91	5.29
repeating key	1	2.91	5.29
repeating key	2	1.51	10.47
repeating key	3	1.17	15.10
repeating key	4	1.06	25.16
full length pseudorandom addend	728751	1.01	29.66

A permutation of the symbols reproduces the measurement to four decimal places, and the boundary is found at a different seat with an identical squared coefficient of variation of 0.2815. Relabeling every symbol changes nothing the measure reads. A one-to-one map has to do that.

A repeating key of length k sends one plaintext symbol to k ciphertext symbols by position, and the mean gap grows with k as that predicts. The measurement is being divided, not destroyed, and by $k = 4$ the split gaps are wider than a statistic over gaps can read. The pattern is still recoverable there by separating the positions that share a key offset, and a ratio near one in those rows is a limit of this measure and not evidence that anything was erased. An attempt to recover it that way is reported in the next paragraph and failed for an unrelated reason.

The last row is different in kind. When the key is as long as the message, the ratio reaches 1.01, the floor the memoryless controls occupy. That is the only case where the structure is absent from the text instead of hidden in it, because it now lives in the key, and it is the condition for perfect secrecy. A one-to-one map cannot remove what this measure reads unless it spends key material equal to the message.

One check on that was attempted and does not support anything. Taking every eighth symbol of the $k = 8$ cipher isolates one key offset, which should leave a plain substitution, and it returns 0.95. The word boundary in this corpus sits at a period near 5.3 symbols, and sampling at a spacing of 8 is above that period. The sampling destroys the boundary regularity on its own, with or without a cipher. The row measures the sampling instead of the transform.

7.4.1 INFORMATION BY FREQUENCY BAND Every result above comes from a detector that returns one symbol, the one whose gaps are most even, and that rejects any candidate occurring less often than once in 64 symbols. Two consequences follow from that, and neither was intended. The occurrence floor admits only frequent symbols. The detector can only ever report

on the commoner symbols. And ranking by evenness treats variation as failure. A symbol whose occurrences cluster is scored as a poor candidate.

Under a Zipf distribution the commoner symbols carry the token count and the rarer ones carry the information, because the surprise of a symbol is $-\log p$ and the many rare symbols each contribute more of it. Scoring every symbol against the same permutation null, instead of only the best one gives the mean ratio over each half of the symbols by frequency:

corpus	commoner half	rarer half
King James Bible 1611 (English)	0.96	0.59
Kivi 1870 (Finnish)	0.97	0.62
Hugo 1862 (French)	0.96	0.69
Austen 1813 (English)	0.95	0.73
Homer, Iliad (Greek)	1.06	0.75
random typing, geometric letter weights	1.00	1.00

The rarer half departs from the shuffle by 0.25 to 0.41 where the commoner half departs by 0.03 to 0.06, and the memoryless control sits at 1.00 in both halves. The departure is a property of the corpora instead of the measure. The direction is below one: the real corpus has gaps more variable than its shuffle. That is clustering, a rare word appearing several times in one passage and nowhere else. Clustering is structure, and the ranking used everywhere above scores it as the opposite.

The averages also separate better than the single winner does. A commoner half average of 0.95 to 1.06 against a control at 1.00 does not separate at all, and every separation reported earlier in this section came from the single best symbol in the commoner half, which has to be found. The rarer half average separates from the control across all five corpora without choosing any symbol.

So the measure that survived Section 7.4 is the weaker of two, and the stronger one was excluded by an occurrence floor added in Section 4.13.0 to reject decoration. Nothing above is withdrawn, since the boundary results stand as measured. The withdrawal covers only the suggestion that the boundary is where the structure sits.

7.4.2 WHAT STAYS THE SAME, AND PERIODIC DISGUISES The transforms in Section 7.4 leave the rarer half measure exactly where the commoner half measure was left, and running them through it describes the measure completely. Cosets are scored by splitting a corpus at a spacing, scoring every coset and averaging, reading all of the text and spending none of the length.

A relabeling of the symbols changes nothing, because the measure reads the gaps between occurrences and a permutation only moves which seat carries which gaps. The single symbol result is identical to four decimals at 0.2815, and over the whole set of symbols a substitution is likewise invisible.

A repeating key of length 8 divides the structure without removing it. Read whole, the ciphertext gives a rarer half figure of 0.99 against 0.73 for the plaintext, a corpus that looks memoryless. Split at a spacing of 8 and averaged over all eight cosets, the ciphertext gives 0.896 and the plaintext gives 0.896. The agreement is exact because each coset was enciphered by one substitution, and the previous paragraph shows the measure cannot see a substitution.

The period does not have to be known. Scanning the spacing over two ciphertexts of the same plaintext, one with a key length of 3 and one with 8, against the plaintext scanned the same way:

spacing	2	3	4	5	6	7	8	9	12	16
plaintext	0.800	0.830	0.876	0.889	0.895	0.895	0.896	0.897	0.896	0.941

key length 3	0.909	0.830	0.935	0.952	0.895	0.975	0.950	0.897	0.896	0.984
key length 8	0.963	1.002	0.913	1.004	0.977	1.010	0.896	1.007	0.958	0.941

Each ciphertext equals the plaintext exactly at every spacing that is a multiple of its key length, at 3, 6, 9 and 12 for the first and at 8 and 16 for the second, and lies above the plaintext everywhere else. The cipher is not producing a dip at the period. It is holding back the plaintext's own structure at every spacing except the ones that line the cosets up with the key, where it holds back nothing at all.

The plaintext row makes that readable, and a scan without it invites two mistakes. The baseline is not flat, climbing from 0.800 at a spacing of 2 to 0.941 at 16 as the subsampling removes structure on its own. A dip has to be judged against that curve and not against 1.0. And the value the coprime spacings take is a property of the key length: with a key of 8 they sit near 1.0 because the structure is divided eight ways, while with a key of 3 they sit between 0.909 and 0.975 because a three way division hides less.

The baseline is not an artifact of the shortening. Each coset holds one spacing's worth of the corpus. A scan compares estimates from different amounts of text and from whatever symbols cleared the occurrence floor at that length. Cutting every coset to 45000 symbols removes both, and the curve still climbs, from 0.769 at a spacing of 2 to 0.936 at 16.

Its shape is $1 - c/\sqrt{s}$ in the spacing s. Fitting the capped curve gives c between 0.237 and 0.277 at eight of the ten spacings measured, with spacings 2 and 12 at 0.327 and 0.343. The exponent follows from what the rarer half measure reads. Its signal is rare symbols clustering into passages, and at a fixed coset length a larger spacing spans proportionally more of the original text. Each cluster contributes $1/s$ as many points to the coset. Deciding a cluster is present is a counting problem, and counting significance grows as the square root of the count. The signal falls as $1/\sqrt{s}$.

That sets what a scan can reach. Structure is not lost at a spacing, it is weakened by a known factor, and a longer corpus buys back any spacing at a quadratic cost.

7.4.3 RANKING FORMAL AND NATURAL LANGUAGES BY STRUCTURE

Sproat's stated limit is that a system with structure and no language passes this test, and every control in Section 7.4 so far has been memoryless, which tests only the other side. The C sources in this repository were measured as a corpus, 422887 bytes over 94 distinct symbols, since a programming language is produced under constraints that share nothing with a vocal tract or a listener decoding in real time.

corpus	commoner half	rarer half
C source	0.59	0.50
Austen 1813 (English)	0.95	0.73
random typing, geometric letter weights	1.00	1.00

The C source departs from the shuffle further than the English does on both halves. So the measure orders these three by how much structure they carry, and a formal language carries more of it than prose does; fixed syntax, repeated identifiers and aligned indentation produce that. The measure is a structure meter. Nothing in it identifies a natural language. That it shows only that a corpus is not memoryless is measured here on the instrument itself, and not taken from the literature.

One reading survives this and is not tested by it. Both corpora that score away from the shuffle were written by people, which leaves the result consistent with the measure detecting production

by a person while failing to tell which kind. Separating those needs a system carrying structure that no person produced, of the sort Rao and colleagues reach for with genomes and protein sequences. No such control is measured here, and no claim of that kind is made.

7.4.4 THE SCALE THE STRUCTURE SITS AT The shuffle used above moves single symbols, which destroys every arrangement at once and cannot say what span an arrangement occupied. Cutting the corpus into blocks of B symbols and shuffling the blocks keeps everything shorter than B and destroys everything longer. Sweeping B locates the signal.

block span	1	8	32	128	512	2048	8192
Austen 1813 (English) rarer half	0.725	0.727	0.729	0.811	0.846	0.934	0.975
C source rarer half	0.500	0.524	0.550	0.623	0.713	0.825	0.915
random typing, geometric letter weights rarer half	0.997	1.001	1.001	0.998	0.999	1.001	1.000

The memoryless control reads 1.0 at every span. The sweep introduces nothing by itself.

The English text is flat from a span of 1 to a span of 32 and then climbs. Blocks of 32 symbols, about six words, destroy its rarer half signal as completely as shuffling single symbols does, and 1.5% of the departure from the shuffle has returned by that point. Its structure begins returning at 128 and is still returning at 8192, which is a passage.

The C source returns 10% of its departure by a span of 32, close to seven times as much, and climbs from a span of 8 upward with no flat region. So the two carry this signal at different spans: the source has arrangements at every scale, including an identifier reused a few lines later, while the prose has almost none of it below a sentence and all of it above one.

That is a statement about the clustering measure and not about the text. The engine workbook measures probes inside a pattern of 24 symbols surviving above the product of the probe rates, at a median ratio of 3.55 over 60 patterns on English prose, which is correlation between positions closer together than 24. The 20992 once quoted here is that workbook's sizing error under the rarest symbol rule. The workbook records that means of this heavy tailed ratio were quoted as typical values, and it still prints 20992 at `:331`, `:1162` and `:1164`. Both hold. The block sweep says the clustering signal has no part below a span of 32, and the probe sequence says a different arrangement lives below 24 that the clustering measure cannot see.

This is the first measurement in this paper that orders a natural language differently from a formal one, and it does so by where the structure sits and not by how much of it there is. A reading of it is available and is not established here: a rare word recurs across a passage because the passage concerns what the word names, which is an arrangement no shorter than the subject being discussed. Nothing in this measurement reaches what a text is about, and the span is the only quantity being reported.

The matching figures for the commoner half move up and down over the same sweep, from 0.945 at a span of 1 to 0.913 at 32 and back to 0.993 at 8192. That half is a mean over many symbols each carrying little structure, which leaves it too diluted to support a claim, and none is made from it.

The lowest point of the scan is the key length in both cases, at 0.830 for the first and 0.896 for the second. Following a doubling ladder finds the second and misses the first, since 3 is not reached by doubling, and the cluster of harmonics carries traps in both directions: a spacing of 12 reads 0.958 against the key of 8 because it shares a factor of 4 with it, which is deeper than the

correct divisor 2 reads at 0.963. Taking the first strong dip while scanning upward returns 12 for a key of 8. The scan has to be read as a whole.

One transform defeats all of it. A pseudorandom addend as long as the message gives 1.004 read whole and 1.004 under every coset scan, because there is no period to split on and no coset in which a fixed substitution was applied. The memoryless controls give 1.005 by the same procedure.

So the measure is unchanged under relabeling, divided but not destroyed by any periodic key, and recoverable from that division without knowing the period. It is defeated only where key material equals message length. That is a complete statement of what it detects, and it is the same statement in both directions: the structure this paper keys on cannot be removed by any transform that a shorter description could undo.

The same finding points the other way for the method. Section 4.13.08 argues that these regularities are regenerated by the conditions that produce a message and are not carried between instances. Sproat's position is the stronger form of that: the structure comes from generators with no intent to communicate at all, dice included. A method keying on a property that nearly every generator produces has a wider reach than one keying on a property special to language, and Section 2.1 never required the source to be a language. Section 2.1 assumes a test of equality between two positions, and the filter needs no regularity in the source. The measures in this section need something regular to have produced the bytes.

7.5 Filtering, coverage and streaming bounds

Navarro [15] requires every filter to be paired with a verification of the positions it could not drop, which is Sections 2.2 and 2.3 for edit distance. The engine workbook records that reading and its quotations.

Nemhauser, Wolsey and Fisher [16] give the $1 - 1/e$ bound for greedy maximization of a monotone submodular function under a cardinality constraint, which Section 2.7 applies. The bibliographic record was checked and the paper was not read. The bound is used in its standard statement.

Clifford, Jalsenius, Porat and Sach [2] prove $\Omega(m)$ bits for the L_1, L_2, L_∞, Hamming, edit and swap distances and for distance functions with wildcard-like properties, pattern matching with character classes among them, with each answer required correct only with constant probability. For other distance functions they give $\Omega(\log m)$ and $O(\log^2 m)$ bits, and exact matching is not among the $\Omega(m)$ cases. Porat and Porat [19] match exactly in $O(\log m \log n)$ bits with Karp-Rabin fingerprints [11], which are Monte Carlo. Both are cited from the engine's record. Those models keep a state while a text passes and permit error. This engine's output grows with the data and its one operation is exact equality, which exposes no arithmetic to fingerprint.

Minsky [13] is cited only as the target of an encoding that is owed (Section 2.7).

8. Conclusion

Whether a permutation null is safe and how much it saves are different claims, and they were not measured as one. The first needs no data and was confirmed over 9,396,207 true matches with

none a false negative. The second is a property of the particular data, and it moves by up to a factor of 6.4 between ways of choosing the probes.

The engine carries the first claim with its conditions in Section 2.7: the count is exact for any valid set of probes once the full comparison runs.

One quantity turned out to do three jobs. The collision probability 2^{-H_2} sets the rate at which a probe chosen with no information leaves a candidate, sets the fraction of alignments a single read rules out, and sets the size of the bias correction that separates the two forms of the estimator. None of the three were derived. All three were measured independently and checked again as correct.

Three experimental results apply to the method:

1. The candidate count is an estimator of collision entropy that needs no histogram or codebook. Its accuracy costs 1 to 20 times against published estimators, depending on the number of probes.

2. Sizing a sequence of probes by multiplying candidate rates leaves too few candidates, by up to four orders of magnitude on structured data, always in the direction of too few.

3. A filter built and run this way rules out $m\,(1-2^{-H_2})$ alignments per read, which is a property of the necessary condition and not of any particular shift table.

Where there is an order on positions, that order is most of the structure there is, and a shift uses it. The candidate rate this paper spends its length measuring is not what decides a search. Section 4.9 records that as a loss of 1.2 to 25 times and then recovers it, first by treating ruling out and shift as one product and then by measuring the shift directly. The arrangement that comes out ahead stores nothing. That follows from the assumptions about the alphabet Σ in Section 2.1.

9. Data and code availability

The benches, the engine, the examples and the hash used as a test oracle are in the orior repository, `https://github.com/dstroy0/orior`. The benches are self-contained C11 programs, compiled with `gcc -O2`. The cost table that ranks a byte is fixed when a bench is built, and every row of the strings bench carries a fingerprint of the 256 costs it was built with, `69c2e2df` for the English profile, since one build reports on one profile only. The estimator bench takes the number of probes as a build parameter, which reproduces the sweep over the number of probes in Section 4.5. The corpora are either generated by the benches or public: the language texts are from Project Gutenberg [21].

Appendix A: The hash used as a test oracle

The SHA-256 implementation used here belongs to the test support and is not reached by the engine. A hash computed by the code under test agrees with that code whatever the code did, and the library is never its own test oracle.

The implementation is checked against five published vectors. RFC 6234 section 8.5 [25] supplies four that sit on the padding boundaries: ``abc'', the 56 octet message whose padding overflows into a second block, 64 octets fed ten times so that padding forms a whole block alone, and

one million `'a'` which pushes the length field past 2^{23} bits over 15,625 compressions. The empty message is the fifth, which RFC 8448 section 3 prints as the TLS 1.3 transcript hash of the empty string.

The standard suites are run offline against stored copies of the published vectors, each copy's source and hash recorded and checked before a vector is read from it.

suite	source	count	what it alone reaches
CAVP ShortMsg	NIST	65	one shot, 0 to 64 bytes
CAVP LongMsg	NIST	64	163 to 6400 bytes
CAVP Monte	NIST	100 × 1000	chained state: each digest feeds the next
CAVP bit ShortMsg	NIST	513	448 do not end on a byte
CAVP bit LongMsg	NIST	512	448 do not end on a byte
CAVP HMAC	NIST	225	150 carry truncated tags
Wycheproof HMAC [33]	C2SP	46	20 modified tags that must not reproduce
streaming splits	invariant	3321 cuts	every split of one message must reach one digest
differential	second implementation	424	padding edges plus random lengths

That is 1554 published vectors from two bodies. Monte earns its place twice over: a hash whose state carries wrongly between blocks passes every one shot table and fails on Monte's first checkpoint. Every vector in every published suite hashes its message in a single call. None reaches the streaming path, and the split and differential rows are the only tests of it.

This is two bodies and not the whole published set of cryptographic test vectors. Widening it is open work.

The general entry takes a 64-bit bit count, because the standard's length field is 64 bits. Trailing bits sit in the high end of the final byte, and the padding marker is set on the first unused bit. There is one padding routine, and the byte and bit entries are that routine with its marker and length filled in differently: the byte case passes the marker `0x80` and a length of eight bits a byte. No second copy exists to drift from the first.

The interface is streaming, begin, take and finish, with the one shot form built on top. It is streaming because a one shot form cannot run the standard's own vectors: one of them is a megabyte, and an ATSAMD51 microcontroller has 192 KB of RAM.

Appendix B: The examples

The examples are 122 programs over nine fields, each one a step of the same pipeline. A field enters at step 1 and leaves at whichever step it has an answer for.

step	what it settles
1. represent	what counts as a part, and how a part is written down
2. partition	how the parts are cut into the cells a distribution is taken over
3. reference	the maximum entropy arrangement the object is measured against
4. measure	the departure from it
5. sift	what the departure keeps or rules out
6. oracle	the outside answer, where a domain has one

field	programs	reaches
language	60	oracle
any corpus	18	sift
crystallography	9	oracle
proteins	8	oracle
cell tracking	7	oracle
art	7	measure
game theory	6	oracle
source code	4	measure
sound	3	measure

Five of these fields carry an oracle step, and the rest stop short of one. This paper's results rest on two oracle answers this work did not produce: crystallography's published cell edges [3] and language's answers extracted by hand. The protein and game theory oracles test against published values too, and those comparisons belong to their own papers. Everything this paper measures beyond the two named is taken against its own permutation null and says so.

2

THE METHOD

2.1 COMPARING TEXTS BY THEIR BYTE PAIRS

This chapter compares texts by how often each pair of adjacent bytes occurs in them. It counts the pairs, measures the distance between two texts, checks the measurement against two halves of the same text, corrects for how many pairs a text uses, groups the texts, and allows for languages that share an ancestor.

1. Counting byte pairs

Take a text T, remove its whitespace, and encode it in UTF-8 as the bytes $b_0 \dots b_{m-1}$. Each pair of neighboring bytes gets one index:

$k_i \;=\; 256\, b_i + b_{i+1}, \qquad k_i \in [0,\, 65536)$

This turns a text of any length, written in 256 possible byte values, into a fixed array of 2^{16} counts. Nothing in it decides what a character is. Every pair gets its own index. No two different pairs share a count.

$P_T(k) \;=\; \frac{c_T(k)}{\sum_j c_T(j)}, \qquad c_T(k) = \left|\{\, i \,:\, k_i = k \,\}\right|$

P_T is a probability distribution over 2^{16} outcomes, the byte-pair histogram of the text.

2. Total variation distance

$D(P, Q) \;=\; \frac{1}{2} \sum_{k=0}^{65535} \left|P(k) - Q(k)\right| \;\in\; [0, 1]$

This is the total variation distance. It is 0 when the two distributions agree everywhere and 1 when no outcome has nonzero probability in both.

3. Split-half check

A distance $D(P_A, P_B)$ between two corpora means nothing until it is known how large a distance chance alone gives at that sample size. Split each corpus in half and measure the distance between its own halves:

$D_{\text{self}}(T) \;=\; D\big(P_{T[0:m/2]},\; P_{T[m/2:m]}\big)$

A distance between corpora can be read when

$D_{\text{self}}(T) \;\ll\; D(P_T, P_{T'}) \quad \forall\, T' \neq T$

and cannot be read when $D_{\mathrm{self}}(T) > \max_{T'} D_{\mathrm{self}}(T')$, because then the corpus is too small to tell apart from noise.

On 20 corpora cut to 6707 bytes each, D_{self} ran from 0.1118 to 0.3192, with a mean of 0.2361.

4. Correcting for how many pairs a text uses

D_{self} falls as a distribution concentrates on fewer pairs, whatever the language. Report how many pairs occur, and the entropy, beside it:

$\mathrm{supp}(P) = \left|\{\, k \,:\, P(k) > 0 \,\}\right|, \qquad H(P) = -\sum_k P(k) \log_2 P(k)$

Across the same 20 corpora, the number of pairs ran from 193 to 970 and rose and fell with D_{self} almost in step. A corpus in an encoding that uses few pairs looks consistent with itself for arithmetic reasons alone. Ranking writing systems by D_{self} ranks them by how many pairs they use, until shown otherwise.

5. Text damaged by extraction

Take the sets of characters $S = \{s_1 \ldots s_q\}$ that only a given writing system uses:

$\mathrm{sets}(T) = \sum_{s \in S} \mathbf{1}\big[\, \exists\, c \in T,\ c \in s \,\big], \qquad \mu(T) = \frac{10^3}{|T_\alpha|} \left|\{\, c \in T \,:\, \mathrm{comb}(c) \,\}\right|$

Here T_α is the text's letters and comb marks combining characters, such as accents written as separate code points. A damaged PDF extraction loses the combining marks first. A file with $\mu \to 0$ and sets $\ll q$ has been damaged even though the text left in it still reads normally.

On 60 extracted papers with $q = 7$, one file scored sets ≥ 5 with $\mu > 0$. Every other file had lost the marks completely.

6. Permutation test

To tell a real split of the data from what splitting any set of keys produces by chance, shuffle the labels within each group the split started from, keeping the size of every cell the same [5]:

$\pi^* \sim \mathrm{Unif}\big(\mathcal{S}(\text{labels within group})\big), \qquad \Delta = \mathrm{E}\big[\, r(\pi^*) \,\big] - r(\pi_{\mathrm{obs}})$

$\Delta \leq 0$ means the observed split gained nothing that a random split of the same shape would not. Over 19 corpora limited to 60000 tokens each, Δ ran from +0.0056 to +0.0901, and the spread of the shuffled splits was at most 0.0133 over 5 draws.

7. Checking the tree against the distances

Group the texts by average-linkage clustering on D. A tree can be built from any distance matrix, including one with no groups in it. Report the cophenetic correlation, the correlation between each measured distance and the height $h(i, j)$ at which i and j first join in the tree [27]:

$\rho_c \;=\; \mathrm{corr}\big(\, D(i,j),\ h(i,j) \,\big)_{i<j}$

A low ρ_c means the tree shows groups the distances do not support, and the result is only an ordering. On 19 languages, over 171 pairs, $\rho_c = 0.8406$.

8. Sampling one language per family

Languages are not independent samples, because related languages share a history. Drawing one language from each family at a time removes that shared part:

$\hat{\pi} = \frac{1}{d}\sum_{t=1}^{d}\frac{1}{|F|}\sum_{f\in F}\mathbf{1}\big[\,h(x_{f,t})\,\big], \qquad x_{f,t}\sim \mathrm{Unif}(f)$

over d draws from the set of families F. Report $\hat{\pi}$ with its standard deviation across the draws. On 418 languages in 121 families with $d = 300$, the rates for each area ran from 0.089 to 0.716, with a spread of 0.029 to 0.069.

3

CHECKING THE RAREST SYMBOL FIRST

This chapter is a conjecture. Comparing a pattern's symbols in order of rarity is an established idea in string search. Sunday's optimal mismatch algorithm compares the pattern's characters in increasing order of how often they occur, so that a mismatch is found as early as possible [30]. In the engine, ordering probes by rarity needs no logarithm and never changes the count. Whether rarity makes the search faster depends on the data, and the engine's table does not separate it out. The table's reorder column ranks probes by how many candidates each one leaves, given the probes already placed, while the engine's rarity order ranks them by rarity alone. Over 65513 alignments with a pattern of length 24, that reorder changes the reads per alignment from 1.877 to 1.875 on the skewed data, where placing probes by candidates left reaches 1.067. It changes them from 1.562 to 1.002 on the synthetic skewed data and from 1.187 to 1.000 on data with period 16. No ordering moves the uniform data from 1.003. Probes are placed by greedy maximum coverage, which drops at least $1 - 1/e$ of what the best set of the same number of probes drops [16]. A rarity order counted on one set of data does not carry over to another.

3.1 THE CONJECTURE

The order in which the probes are checked is the only choice left to make, and the goal is to drop a candidate in as few checks as possible. Ordering the probes by the frequency of their symbols in the data makes that choice.

Take a pattern with three positions, $A = \{a_1, a_2, a_3\}$, whose symbols have these probabilities in the data:

position	symbol	probability
a_1	common	0.40
a_2	medium	0.08
a_3	rare	0.001

Checked in order of rarity, the search compares a_3 first, then a_2, then a_1.

A common symbol is a poor first check. The data holds it often. Most candidates pass it. A rare symbol is a good first check, because almost every candidate fails it at once. The chance that the first check drops a candidate is

$$\Pr(\text{dropped at the first check}) = 1 - P\big(x(a_{\text{rare}})\big).$$

If the rarest symbol has probability 0.001, the first comparison drops 99.9% of all candidates.

The order changes nothing about which places are matches. Each check is still exact, and failing any check still rules a place out for good, $\neg\,\mathrm{Anc}_A(s) \Rightarrow \neg\,\mathrm{Occ}(s)$. The frequencies only decide which check runs first. The data's own statistics steer the search, and the rarest probes clear most of the candidates before the processor reaches the loop over them.

4

TERMS

4.1 THE WORDS THIS PAPER USES

This chapter says what each technical word in this research paper means, using the name the field already gives it.

Most of the words here are standard and need no explanation. This chapter covers the rest: words with a narrow meaning in this paper, and ideas whose field name a reader might not expect. Where a field has a name for an idea, the paper uses that name and cites where it comes from.

Sets and arrangements

alphabet. The set of symbols a pattern and a text are written in, written Σ. The paper assumes nothing about it: neither order, arithmetic nor a promise that it is finite or can be listed. When the structure of the data defines a topology on it, that topology is written τ_Σ.
configuration. A finite arrangement of points or offsets. It is the shape of one pattern when order does not matter. The alphabet supplies the symbols a pattern could hold, and the configuration says which positions the pattern uses.
family. A collection whose members are themselves sets or configurations, such as every choice of probe positions, or every shuffled copy of the data.
orbit. Everything one configuration becomes under a stated group of moves, such as rotations or translations. Two configurations share an orbit only once the group and how it acts are stated.
residue class. A congruence class modulo a stated number. It is never used here to mean a bucket, a cluster or a grid cell.
subconfiguration. A subset $A \subseteq D$ of a pattern's positions, used as a filter. Checking only these positions keeps the pattern's structure and drops some of its conditions.
support. The finite set of positions a pattern occupies, written $D \subset G$, or the set of outcomes a distribution gives nonzero probability. Both are the standard meanings of the word.

The permutation null

permutation test. A test that builds its null distribution from the data itself, by shuffling it many times and computing the same statistic on every shuffled copy [5]. Here the shuffles keep the count

of every symbol and any stated grouping. If Π_Σ is the set of those shuffles, then for a statistic M the quantity this paper reports is

$$\Delta_\Sigma[M] = M(x) - \mathbb{E}_{\pi\sim\Pi_\Sigma}\big[M\big(\pi(x)\big)\big]\,,$$

the observed statistic minus its mean over the shuffled copies. It is a difference from a null and not a new operator. It says what the arrangement of the data adds beyond its symbol counts, and it does not say what produced the arrangement.

maximum entropy reference. The shuffled copies are the most disordered arrangements the symbol counts allow. Building a reference that way is Jaynes's maximum entropy principle [9]. The reference uses the counts and assumes nothing else. Entropy is strictly concave and a count constraint is linear. Exactly one distribution meets the constraint, and the reference is unique.

free energy above equilibrium. The distance from a measured state to its own maximum entropy state under a constraint. Thermodynamics gives that quantity this name. The engine workbook [22] records the match as exact, following Parrondo, Horowitz and Sagawa [18].

relative entropy. Also called Kullback-Leibler divergence and written `D(p||q)`, a distance between two distributions [12]. The match to free energy holds for the whole reference and does not carry over to each statistic. Section 4.3.1 of the first chapter shows that what decides a probe order's worth is not a relative entropy, and the engine workbook records the same [22]. The tools compute either a spread of gaps against shuffled copies, or a standardized residual against pooled counts. Both measure a difference from a maximum entropy reference, and neither is a relative entropy.

standardized residual against a pooled null. A test that pools the counts of every run, spreads them evenly, and scores each run as its own binomial. A standardized residual is the observed count less the expected, over the square root of the expected [6].

rarer half, commoner half. The symbols split at the median of their frequencies. Every difference from the null reported in the engine workbook [22] is over the rarer half unless it says otherwise.

Searching

filter and verify. The search this whole paper rests on. Check a few positions of the pattern first, drop any place in the text that fails one of them, and run the full comparison only on the places left. A filter that can never drop a true match is called lossless, and filtering followed by verification is standard in string matching [15]. Here the positions to check can be any subset of the pattern, chosen in any order.

probe. One position of the pattern checked by the filter. A place in the text whose symbol at that offset differs from the pattern's cannot be a match. A probe is a necessary condition for a match and never a sufficient one.

probe sequence. Probes applied one after another, each one checking only the places the previous ones kept. The depth $\log_2 N/H_2$ appears throughout, and means the number of probes an independence model predicts before one candidate is left. The engine places at most four. How many it places depends on the number asked for, the candidates available and the text.

candidate. A place in the text that every probe so far has kept. It can still turn out not to match.

false negative. A true match the filter drops. A lossless filter has none, and every bench row in this paper counts them.

expected comparisons before a mismatch. How far into a pattern a mismatch is expected to appear, $m(1 - 2^{-H_2})$, for a pattern of length m.
equality test. In the engine, the function the caller supplies to say whether two positions hold the same symbol. The engine never reads a symbol any other way.
test oracle. In the corpus work, a hand extraction of what a paper holds, used as the answer a tool is graded against. This is the software testing term [8]. The hand extractions also work the other way: they define what is known. Anything a tool finds outside them belongs to what is not yet known. Written languages with long recorded rule sets, such as Chinese, put scene and subject into one unit and state the writer's meaning directly. The conjecture is that such languages are easier to sort by family and by author, and that tracing why could show a wider boundary. Nothing here measures that.

Corpora

reference corpus. The corpus a target is compared against, in the corpus linguistics sense. In classification it is the negative class, and in signal detection it is the background.
gold standard corpus. A corpus checked by hand so that it can be trusted as correct [32]. Here it is the output holding only speech in the target language, with reconstructions, rejected forms, other languages and analysis lines removed. It is checked in two directions against a hand extraction, and the Salishan research paper lists the papers in it [23].
extraction error classes. The kinds of damage a PDF text extraction does, recorded for each paper to trace an error to its source. There are five: substitution, collapse, transposition, insertion and deletion. The classes that can be repaired are repaired automatically.
split-half estimate. The distance between two halves of one corpus, used as the smallest difference a comparison between corpora has to beat. Splitting one set of measurements in two and comparing the halves is the split-half method [28, 1]. Cutting a corpus at its midpoint measures the order its papers are in, not the resolution, as the Salishan research paper shows [23]. The halves are therefore sampled by alternating.
byte-pair histogram. A text reduced to the counts of each pair of adjacent bytes. In the field this is a bigram distribution, the digram approximation of Shannon's paper [26]. A broader idea sits behind it, and nothing here measures it. An object spread over many dimensions cannot be known in full without measuring it, and nothing measures the infinite directly. What can be observed is how far the object is from its own most disordered state, because that state is fixed by what the object is made of. Flattening the object to a set of counts keeps that distance and loses the rest, and the counts are never cut off at a bound, because the object's own makeup already sets its limits.

Measurements

The two measurements below are different, and a row measured with one cannot stand in for a row measured with the other. Both rest on Section 2.2 of the first chapter.
the permutation null. The difference between an arrangement and the shuffled copies of the same symbols. Each copy is a Fisher-Yates shuffle [4]. It carries most of the results in the engine workbook [22], and no outside source has the answer to check it against. The conjecture behind it, which nothing here measures, is that the structure in Σ shuffles what the null set null_0 could carry, and that the shuffled order shows what happens to null_0.

the shift agreement detector. Finds a period or an offset by how often the data agrees with a shifted copy of itself. Published cell edges in the Crystallography Open Database [3] check it three separate ways.

BIBLIOGRAPHY

[1] W. Brown. *Some Experimental Results in the Correlation of Mental Abilities*. British Journal of Psychology 3(3) 296-322, 1910, doi:10.1111/j.2044-8295.1910.tb00207.x.

[2] R. Clifford, M. Jalsenius, E. Porat, B. Sach. *Space Lower Bounds for Online Pattern Matching*. CPM 2011, arXiv:1106.4412, doi:10.48550/arXiv.1106.4412, submitted 22 June 2011.

[3] Crystallography Open Database. https://www.crystallography.net/cod/.

[4] R. A. Fisher, F. Yates. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver and Boyd, Edinburgh. 1938.

[5] P. Good. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*, third edition. Springer, New York, 2005, ISBN 978-0-387-20279-2, doi:10.1007/b138696.

[6] S. J. Haberman. *The Analysis of Residuals in Cross-Classified Tables*. Biometrics 29(1) 205-220, 1973.

[7] Horspool, R. Nigel. *Practical Fast Searching in Strings*. Software-Practice and Experience 10(6) 501-506, doi:10.1002/spe.4380100608. 1980.

[8] W. E. Howden. *Theoretical and Empirical Studies of Program Testing*. IEEE Transactions on Software Engineering 4(4) 293-298, 1978.

[9] Jaynes. *Information Theory and Statistical Mechanics*. Physical Review 106(4) 620-630, 15 May 1957.

[10] M. Kac. *On the notion of recurrence in discrete stochastic processes*. Bulletin of the American Mathematical Society 53(10) 1002-1010, doi:10.1090/s0002-9904-1947-08927-8.

[11] R. M. Karp, M. O. Rabin. *Efficient randomized pattern-matching algorithms*. IBM Journal of Research and Development 31(2) 249-260. 1987.

[12] S. Kullback. *Information Theory and Statistics*. John Wiley and Sons, 1959.

[13] M. L. Minsky. *Recursive Unsolvability of Post's Problem of "Tag" and Other Topics in Theory of Turing Machines*. Annals of Mathematics, second series, 74(3) 437-455, JSTOR 1970290. 1961.

[14] Montemurro. *Universal Entropy of Word Ordering Across Linguistic Families*. PLoS ONE 6(5):e19875, 2011.

[15] G. Navarro. *A Guided Tour to Approximate String Matching*. ACM Computing Surveys 33(1) 31-88, 2001.

[16] Nemhauser, Wolsey, Fisher. 1978.

[17] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, M. Boyle. *Recommendation for the Entropy Sources Used for Random Bit Generation.* NIST SP 800-90B, doi:10.6028/nist.sp.800-90b. 2018.

[18] J. M. R. Parrondo, J. M. Horowitz, T. Sagawa. *Thermodynamics of information.* Nature Physics 11, 131 (2015).

[19] B. Porat, E. Porat. *Exact and Approximate Pattern Matching in the Streaming Model.* FOCS 2009, pages 315-323.

[20] E. L. Post. *Formal Reductions of the General Combinatorial Decision Problem.* American Journal of Mathematics 65(2) 197-215. 1943.

[21] Project Gutenberg. https://www.gutenberg.org.

[22] D. Quigg. *Workbook.* orior research papers, https://github.com/dstroy0/orior. 2026.

[23] D. Quigg. *Salishan.* orior research papers, https://github.com/dstroy0/orior. 2026.

[24] A. Rényi. *On measures of entropy and information.* Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability 1, 547-561. 1961.

[25] D. Eastlake, T. Hansen. *US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF).* RFC 6234, doi:10.17487/rfc6234. 2011.

[26] C. E. Shannon. *A Mathematical Theory of Communication.* Bell System Technical Journal 27, 379-423 and 623-656, July and October 1948.

[27] R. R. Sokal, F. J. Rohlf. *The Comparison of Dendrograms by Objective Methods.* Taxon 11(2) 33-40, 1962.

[28] C. Spearman. *Correlation Calculated from Faulty Data.* British Journal of Psychology 3(3) 271-295, 1910, doi:10.1111/j.2044-8295.1910.tb00206.x.

[29] R. Sproat. *Ancient Symbols, Computational Linguistics, and the Reviewing Practices of the General Science Journals.* Computational Linguistics 36(3) 585. 2010.

[30] D. M. Sunday. *A Very Fast Substring Search Algorithm.* Communications of the ACM 33(8) 132-142, 1990.

[31] Y. Tsai. *Average Case Analysis of the Boyer-Moore Algorithm.* Random Structures and Algorithms 28, 481 (2006).

[32] L. Wißler, M. Almashraee, D. Monett Díaz, A. Paschke. *The Gold Standard in Corpus Annotation.* IEEE German Student Conference (IEEE GSC), 2014.

[33] C2SP. *Project Wycheproof.* https://github.com/C2SP/wycheproof.

[34] Zanette. *Universal Entropy of Word Ordering Across Linguistic Families.* PLoS ONE 6(5):e19875, 2011.

[35] G. K. Zipf. *Human Behavior and the Principle of Least Effort.* Addison-Wesley, Cambridge, Massachusetts. 1949.